

Unit 1: Introduction to android and it's tool chain

1.1 Architecture Of Android OS

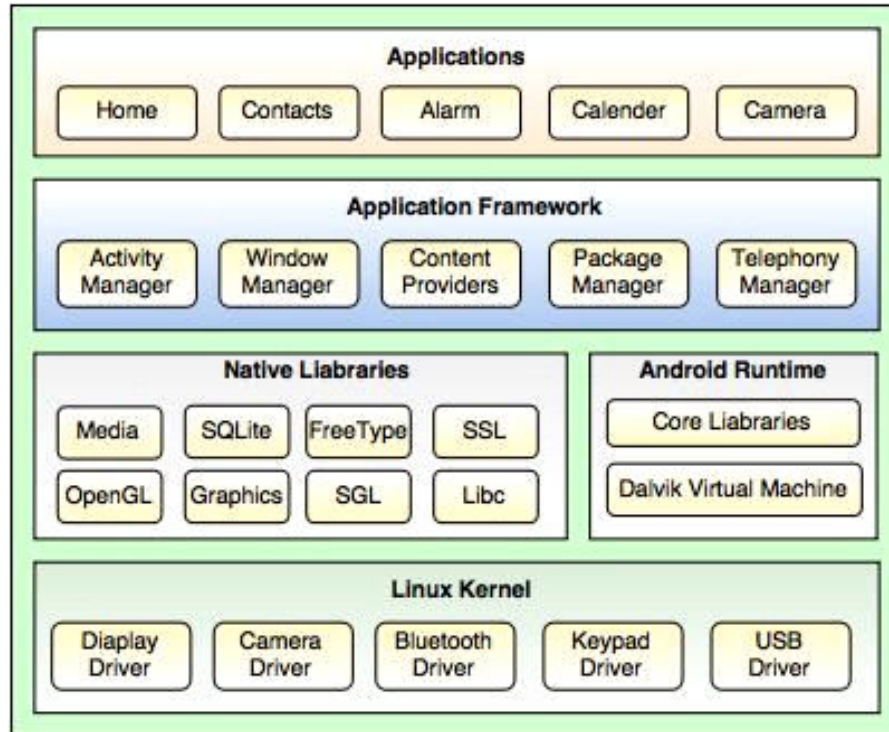


Fig. Android Architecture

Android operating system is built using a layered architecture that includes different components. These layers help Android run apps, manage hardware, and provide a user-friendly experience. The Android architecture is divided into four main layers:

1. Linux Kernel (Bottom Layer):

- This is the core part of Android OS.
- It communicates directly with the hardware (like camera, Wi-Fi, Bluetooth).
- It also handles security, device drivers, memory management, and power management.
- Android uses Linux Kernel version customized by Google.

2. Hardware Abstraction Layer (HAL):

- HAL acts as a bridge between hardware and software.
- It provides standard interfaces for hardware (like audio, camera) so that Android doesn't need to worry about how each device works internally.
- Example: Different phones have different cameras, but HAL makes them work in a common way.

3. Android Runtime (ART) & Libraries:

- Android Runtime (ART):

- It helps Android apps run.
- Each app runs in its own virtual machine for better security and performance.
- Core Libraries:
 - These provide functions for app development like data storage, web browsing, and graphics.

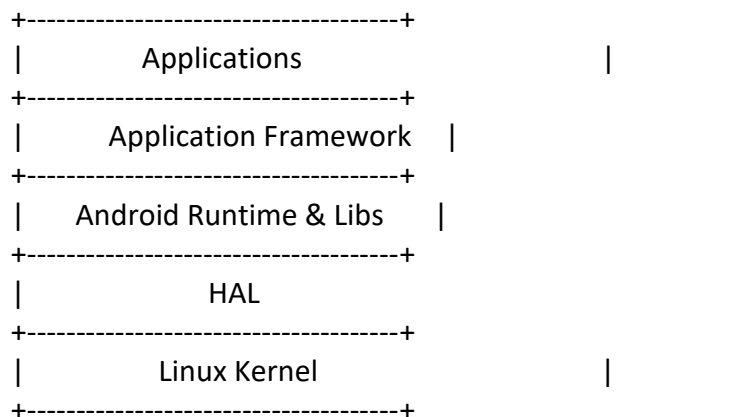
4. Application Framework:

- This layer provides building blocks (like Activity, Service, Content Provider) for app development.
- It helps developers create apps that can interact with the system and other apps.
- It includes:
 - Activity Manager: Manages app activities.
 - Notification Manager: Manages status bar notifications.
 - Content Providers: Shares data between apps.

5. Applications (Top Layer):

- These are the apps that users interact with.
- Examples: Phone, Messages, WhatsApp, Instagram, etc.
- These apps are built using the Application Framework and run on top of Android Runtime.

Diagram :



1.2 Android Development Tools

Android Development Tools (ADT) are the software tools that help developers to create, test, and debug Android applications. These tools make it easier to build high-quality apps that can run on different Android devices.

Main Android Development Tools:

1. Android Studio:

- Official IDE (Integrated Development Environment) for Android development.
- Provides code editor, layout designer, and emulator.

- Helps you write, test, and run your app in one place.

2. Android SDK (Software Development Kit):

- A collection of libraries and tools to build Android apps.
- Includes APIs, emulator, and debugging tools.

3. Emulator:

- A virtual Android device that runs on your computer.
- Used to test apps without a real phone.

4. ADB (Android Debug Bridge):

- A command-line tool that helps communicate with an Android device.
- Useful for installing apps, logging errors, and debugging.

5. Gradle:

- A build automation tool.
- Helps in building, packaging, and managing dependencies for Android projects.

1.2.1. Android SDK and SDK Manager

- SDK stands for Software Development Kit.
- The Android SDK is a collection of tools, libraries, and code samples that developers use to build Android apps.

✓ Main Components of Android SDK:

1. Libraries: Pre-written code to help with things like camera, GPS, etc.
2. Emulator: A virtual phone for testing apps.
3. ADB (Android Debug Bridge): Tool for connecting your computer to your Android device.
4. Build Tools: Helps in compiling and packaging apps.
5. APIs: Allow your app to use Android features like calling, messaging, and sensors.

◆ Why is Android SDK Important?

- It provides everything needed to create Android apps.
- Ensures your app works on different Android versions and devices.
- Helps in testing and debugging apps easily.

◆ What is SDK Manager?

The SDK Manager is a tool in Android Studio that helps you download, install, and update SDK packages.

✓ Features of SDK Manager:

1. 📄 Download Android versions (API levels)

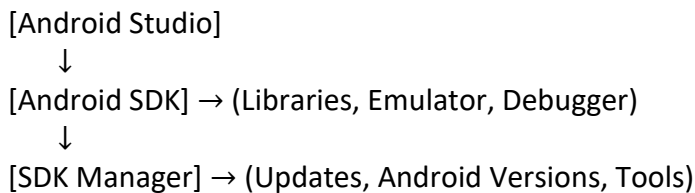
- You can select which version (like Android 9, 10, 11, etc.) you want to develop for.
2.  Update tools and libraries
Keeps your development tools up to date.
 3.  Install additional tools
Like emulator images, Google Play services, or USB drivers.

How to Access SDK Manager?

In Android Studio:

Go to Tools > SDK Manager – here, you can see and manage all your SDK packages.

Simple Diagram



Conclusion:

The Android SDK gives you everything needed to build Android apps, while the SDK Manager helps you manage and update these tools easily. Together, they make Android app development faster, smoother, and more powerful.

1.2.2. The Android Virtual Device, Emulator

What is Android Virtual Device (AVD)?

- An Android Virtual Device (AVD) is a virtual mobile device that runs on your computer.
- It is used to test and run Android apps without needing a real smartphone.

Features of AVD:

- Acts like a real Android phone or tablet.
- Can be set up with different screen sizes, Android versions, RAM, etc.
- You can test how your app works on different types of devices.

What is Android Emulator?

- The Android Emulator is a tool that runs the AVD on your computer.
- It shows a virtual Android device on your screen where you can open, install, and test apps — just like a real phone.

Features of Emulator:

- You can click, swipe, and type just like on a real phone.
- Supports camera, GPS, calls, SMS, and more.
- Useful for testing apps on multiple Android versions and devices.

How to Use AVD and Emulator in Android Studio:

1. Open Android Studio.
2. Go to Tools > Device Manager.
3. Click on Create Virtual Device.
4. Choose phone model, Android version, and other settings.
5. Click Finish, then click Play to launch the Emulator.

Benefits of Using AVD and Emulator:

- No need to use a real phone during development.
- Saves time and helps test on multiple devices and Android versions.
- Easier to debug and monitor performance.

Conclusion:

- AVD is a virtual version of an Android device, and the Emulator is the tool that runs it.
- They help developers to test and develop apps easily without a physical phone, making the app development process faster and more efficient.

1.2.3. Dalvik Debug Monitor Service (DDMS)

What is DDMS?

- DDMS (Dalvik Debug Monitor Service) is a debugging tool provided in Android Studio (earlier in Eclipse) that helps developers to test and monitor Android apps during development.
- It works with the Dalvik Virtual Machine (DVM), which was used to run Android apps before Android 5.0 (Lollipop).

 Think of DDMS like a control panel that lets developers watch how their app is working in real time.

Main Features of DDMS:

1.  Device Monitoring:
 - Shows a list of connected Android devices and emulators.
 - You can choose which device to debug.
2.  Logcat Viewer:
 - Displays system messages and errors.
 - Helps find and fix bugs in your app.
3.  Thread & Heap Information:
 - Shows all running threads in the app.
 - Shows how much memory the app is using (heap size).
4.  Screen Capture:

- Takes screenshots of the emulator or connected device screen.

5. 📁 File Explorer:

- Allows you to browse files and folders inside your app's data directory.

6. 📍 Location Simulation:

- You can simulate GPS data (like setting a fake location for testing map apps).

7. ➡️📱 Send SMS & Call:

- You can test how your app behaves when it receives SMS or calls.

● Note:

- DDMS worked with the Dalvik VM (used in older Android versions).
- In newer Android versions (ART – Android Runtime), DDMS features are included inside Android Studio tools like Logcat, Profiler, and Device File Explorer.

⬅️ END Conclusion:

DDMS was a powerful tool for developers to debug, monitor, and test Android apps. It allowed real-time tracking of memory usage, errors, logs, and device actions. Though it was used mainly in older Android versions, many of its features still exist today in Android Studio under different names.

1.2.4. Android Debug Bridge (ADB)

◆ What is ADB?

- ADB (Android Debug Bridge) is a command-line tool that allows developers to communicate with Android devices (real or virtual) from a computer.
- It helps in debugging apps, transferring files, installing APKs, and more — using simple commands from the terminal or command prompt.

💡 Think of ADB as a remote control for your Android device from your PC.

✓ Main Uses of ADB:

1. 📱 Connect to Devices:

- You can connect your Android phone or emulator to your PC using ADB.

2. 📁 Install/Uninstall Apps:

- Install APKs with:
`adb install appname.apk`
- Uninstall apps with:

adb uninstall com.example.app

3. Debug Applications:

- See log messages, errors, and crashes using:
adb logcat

4. Transfer Files:

- Move files between your computer and Android device:
 - adb push file /sdcard/ (to send)
 - adb pull /sdcard/file (to receive)

5. Access Shell (Terminal):

- Use: adb shell
This gives direct access to your device's command-line.

6. Take Screenshots or Record Screen:

- adb shell screencap /sdcard/screen.png
- adb shell screenrecord /sdcard/video.mp4

 How to Use ADB:

1. Enable Developer Options and USB Debugging on your Android device.
2. Connect your phone via USB cable.
3. Use Command Prompt (Windows) or Terminal (Mac/Linux).
4. Run commands like:
 - adb devices (to check connection)
 - adb install, adb logcat, etc.

 Conclusion:

ADB is a powerful tool that allows developers to control Android devices, install apps, debug errors, and test features without using the touchscreen. It makes Android app development and testing much easier and faster.

Unit 2: Android - Key Components

2.1 Components Of Android Application - Activities, Services, Broadcast Receivers, Content Providers

Android apps are made of 4 core building blocks (components) that define how they work:

- Activity → A single screen with a user interface (e.g., Login Screen, Home Screen).
- Service → Runs in the background (e.g., music player, downloads).
- Broadcast Receiver → Responds to system-wide events (e.g., low battery, incoming call).
- Content Provider → Manages shared app data (e.g., contacts, storage access).

Example:

A WhatsApp app uses:

- Activities (Chat screen, Status screen),
- Services (message notifications in background),
- Broadcast Receivers (detect new messages),
- Content Providers (access phone contacts).

Components of Android Application – Activities

◆ What is an Activity?

- An Activity is one screen of an Android app that the user can see and interact with.

💡 Think of an Activity like a page in a book — each page (activity) shows different content, like a login screen, home screen, or settings screen.

✓ Key Points about Activities:

1. 🏠 UI-Based Component:

- An Activity provides the user interface (UI) for users.
- It displays things like buttons, text, images, and allows user input.

2. 🔄 One App = Multiple Activities:

- Most apps have more than one Activity.
- Example: A shopping app may have:
 - Login Activity
 - Home Activity
 - Product Details Activity
 - Cart Activity

3. 🔄 Activity Lifecycle:

- Activities go through different stages like:
 - onCreate() – when the activity is first created.
 - onStart(), onResume() – when the activity becomes visible and interactive.
 - onPause(), onStop(), onDestroy() – when the activity is closed or hidden.
- These lifecycle methods help manage memory, user actions, and app performance.

4. 🔄 Switching Between Activities:

- You can move from one activity to another using Intent.
- Example:

```
Intent i = new Intent(this, SecondActivity.class);
startActivity(i);
```

5. 📄 Defined in AndroidManifest.xml:

- Every activity used in your app must be declared in the AndroidManifest.xml file.

✓ Example:

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

This is a basic Activity that loads a layout named activity_main.

← Conclusion:

An Activity is one of the most important parts of an Android app. It represents a single screen with which users can interact. Managing activities properly is essential for building smooth, user-friendly Android apps.

Components of Android Application – Services

◆ What is a Service in Android?

- A Service is a component of an Android application that runs in the background to perform long-running tasks without showing any user interface.

💡 Think of a Service like a background worker that keeps running even if the user is not looking at the app.

✓ Key Features of Services:

1. ⌚ Runs in Background:

- Services are used for tasks that should continue even if the user switches to another app.
- Example: Playing music, downloading files, checking messages.

2. ✕ No User Interface (UI):

- Unlike Activities, Services do not have any screen or buttons.
- It works silently behind the scenes.

3. 🔗 Works with Other Components:

- Services can be started by Activities, Broadcast Receivers, or other Services.

4. ⚙️ Can Run Continuously:

- You can make a Service keep running until the task is finished or the app is closed.

🔧 Types of Services:

1. Started Service:

- Starts when a component (like an Activity) calls startService().

- Runs in the background until stopped by stopService() or when the task is complete.

2. Bound Service:

- Connects (binds) with other components like Activities.
- Runs as long as the component is connected to it.

3. Foreground Service:

- Shows a notification and keeps running even when the app is closed.
- Example: Music apps like Spotify use Foreground Services.

✓ Example Code:

```
public class MyService extends Service {
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // Task running in background
        return START_STICKY;
    }

    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

✓ Declared in AndroidManifest.xml:

Every service must be declared in the manifest file like this:

```
<service android:name=".MyService" />
```

←_{END} Conclusion:

A Service is a background component used to perform long tasks like music playing, file downloading, or location tracking without showing any screen. It helps improve the user experience by handling tasks efficiently in the background.

Components of Android Application – Broadcast Receivers

◆ What is a Broadcast Receiver?

- A Broadcast Receiver is a component in Android that listens for messages (called broadcasts) sent by the system or by other apps.

💡 Think of it like a radio that listens for announcements (like battery low, message received, or app installed) and reacts when it hears them.

✓ Why Use Broadcast Receivers?

Broadcast Receivers allow apps to:

- React to system events like:

- Battery low 🔋
- Wi-Fi connected 📶
- SMS received 📧
- Phone rebooted 🔄
- Respond to custom messages sent from other parts of your app or other apps.

✓ Key Features:

1. 🚫 No UI (User Interface):

Broadcast Receivers do not have any screen. They run in the background.

2. ⌚ Runs for a Short Time:

When an event occurs, the receiver wakes up, handles it quickly, and then shuts down.

3. 📄 Declared in Manifest or Registered in Code:

You can register a Broadcast Receiver in:

- AndroidManifest.xml (for system-wide events)
- Java/Kotlin code (for events while the app is running)

🔧 Example of System Broadcasts:

- android.intent.action.BOOT_COMPLETED – when the device finishes booting.
- android.net.conn.CONNECTIVITY_CHANGE – when internet connectivity changes.

✓ Example Code:

```
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        // Code to execute when broadcast is received
        Toast.makeText(context, "Broadcast Received!",
            Toast.LENGTH_SHORT).show();
    }
}
```

✓ In AndroidManifest.xml:

```
<receiver android:name=".MyReceiver">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
    </intent-filter>
</receiver>
```

🏠 Conclusion:

A Broadcast Receiver allows Android apps to listen for and respond to system-wide or app-specific events. It helps build smart, responsive apps by reacting to changes in the device or system environment in real time.

◆ What is a Content Provider?

- A Content Provider is a component in Android that is used to share data between different apps.

💡 Think of it like a gatekeeper that manages access to your app's data and lets other apps read or modify it (if permission is given).

✓ Why Use Content Providers?

- To store and manage data (like contacts, messages, files, etc.)
- To share data with other apps securely.
- To access system data like contacts, images, videos, etc.

📁 Example:

- Your app wants to access the user's contacts. You can use the Contacts Content Provider to read contact details from the phone.

✓ Features of Content Providers:

1. 🗄️ Handles Structured Data:

- Stores data in a table-like format (rows & columns).
- Works similar to a database.

2. 🔒 Provides Secure Access:

- Other apps can access your data only if you allow it using permissions.

3. 🌐 Uses Content URIs:

- Every Content Provider uses a Content URI to access data.
- Example: `content://contacts/people`

4. ➡️ Used by System Apps:

- The Contacts app, SMS app, Media app, etc., all use Content Providers.

✓ Basic Functions (CRUD Operations):

Method	Use
<code>query()</code>	Read data
<code>insert()</code>	Add new data
<code>update()</code>	Modify existing data
<code>delete()</code>	Remove data

✓ Example Code to Access Contacts:

```
Cursor cursor = getContentResolver().query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null);
```

```
while (cursor.moveToNext()) {
    String name =
```

```

        cursor.getString(cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME
    ));
}

```

✓ Declare in AndroidManifest.xml:

```

<provider
    android:name=".MyContentProvider"
    android:authorities="com.example.provider"
    android:exported="true" />

```

✚ Conclusion:

Content Providers are used to store, access, and share app data in a structured and secure way. They are especially useful when you want to let other apps read/write your app's data or when you need to access system data like contacts or images.

2.2 Directory Structure of Android Application

■ When you create a new Android project, you'll see folders like:

◆ 1. app/ Folder:

This is the main folder that holds all your app's code, resources, and configuration files.

Inside app/, we have:

✓ a) java/

- Contains all Java/Kotlin code for:
 - Activities
 - Fragments
 - Services
 - Adapters
 - Utility classes

💡 You may also see com.example.myapp or similar package names here.

✓ b) res/ (resources)

- Stores non-code files like:
 - layout/ – XML files for UI design (e.g., activity_main.xml)
 - drawable/ – Images (PNG, JPG, XML shapes)
 - values/ – Strings, colors, themes (in strings.xml, colors.xml)
 - mipmap/ – App launcher icons
 - menu/ – XML files for app menus

✓ c) AndroidManifest.xml

- Describes the structure and settings of the app.
- Declares:
 - Activities
 - Services
 - Permissions

- Content Providers

◆ 2. gradle/ Folder

- Contains configuration files for Gradle, the build system.
- Includes wrapper files like gradle-wrapper.properties.

◆ 3. build.gradle Files

There are two:

- Project-level build.gradle: General settings (like Gradle version, repositories)
- App-level build.gradle: App-specific settings (like dependencies, SDK version)

◆ 4. .idea/ and .gradle/ Folders (auto-generated)

- Used by Android Studio for project settings and build info.

◆ 5. build/ Folder

- Stores the compiled files (auto-generated). Usually hidden and recreated during builds.

✦ Example Directory Structure:

MyApplication/

```

|
├── app/
|   ├── src/
|   |   ├── main/
|   |   |   ├── java/com/example/myapp/
|   |   |   ├── res/
|   |   |   |   ├── layout/
|   |   |   |   ├── drawable/
|   |   |   |   ├── mipmap/
|   |   |   |   └── values/
|   |   └── AndroidManifest.xml
|
├── gradle/
├── build.gradle (Project level)
└── settings.gradle

```

2.2.1. AndroidManifest.xml

◆ What is AndroidManifest.xml?

- The AndroidManifest.xml file is a very important file in every Android project.
- It gives essential information about the app to the Android system.

💡 Think of it like an identity card or blueprint of your app that tells the system what your app can do.

✓ Main Purposes of AndroidManifest.xml:

1. 📄 Declares App Components:

- Lists all Activities, Services, Broadcast Receivers, and Content Providers used in the app.

2. 🔒 Permissions:

Requests permissions like:

- Internet access
- Read contacts
- Use camera

Example:

```
<uses-permission android:name="android.permission.INTERNET" />
```

3. 🕒 Defines the Main Activity:

- Tells which screen (Activity) should open first when the app starts.
- Uses intent-filter with MAIN and LAUNCHER actions.

4. 📦 Declares App Features and Requirements:

- Specifies minimum SDK version, hardware features, themes, etc.
- Example:

```
<uses-sdk android:minSdkVersion="21" android:targetSdkVersion="33" />
```

5. 📁 Gives App Name, Icon & Theme:

Sets app-level info like:

- App icon
- App name
- App theme
- App package name (unique ID)

✓ Basic Example of AndroidManifest.xml:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication">
```

```
    <uses-permission android:name="android.permission.INTERNET" />
```

```
    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MyApp">
```

```
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
```

```
</activity>

</application>
</manifest>
```

← END Conclusion:

The AndroidManifest.xml file is a central configuration file that defines the structure, permissions, components, and metadata of the app. Without this file, the Android system won't know how your app is organized or how it should behave.

2.2.2. Layouts & Drawable Resources

◆ What is a Layout in Android?

- A Layout in Android is used to design the User Interface (UI) — it tells how the views (like buttons, text, images) will appear on the screen.

💡 Think of a layout as a blueprint or structure of the screen — it arranges the views properly.

✓ Types of Layouts in Android:

1. LinearLayout

- Arranges views in a single row or column (either vertically or horizontally).
- Example:

```
<LinearLayout
    android:orientation="vertical">
    <TextView />
    <Button />
</LinearLayout>
```

2. RelativeLayout

- Positions views relative to each other or to the parent.
- Example: Button below TextView, aligned to center, etc.

3. ConstraintLayout

- A flexible and powerful layout that lets you connect views with constraints (best for complex UIs).
- Example: Place a button exactly in the center of the screen.

4. FrameLayout

- Displays one view at a time, useful for overlapping views.

5. TableLayout

- Organizes views into rows and columns, like a table.

6. GridLayout

- Arranges views in a grid form, good for calculator-type UIs.

✓ How Layouts are Written:

- Layouts are usually written in XML files.
- File location: res/layout/
- Example file: activity_main.xml

✓ Example Layout XML:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="Welcome!"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <Button
        android:text="Click Me"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

✦ Important Attributes Used in Layouts:

Attribute	Description
layout_width	Width of the view (match_parent, wrap_content)
layout_height	Height of the view
orientation	Direction of view (horizontal/vertical) in LinearLayout
id	Unique name to access the view in Java/Kotlin
padding / margin	Space inside / outside the view

← END Conclusion:

Layouts are the backbone of Android UI design. They arrange the elements on the screen and define how the app looks and feels. Understanding different types of layouts helps create beautiful and responsive Android apps.

✓ Drawable Resources in Android

◆ What are Drawable Resources?

- In Android, drawable resources are images or graphics used in the app.
- They are stored in the res/drawable/ folder of your Android project.

💡 Think of drawables like the visual elements of your app — such as logos, buttons, backgrounds, or icons.

✓ Why Use Drawable Resources?

- To manage all images in one place
- To support multiple screen sizes
- To easily apply images and shapes to UI elements like buttons, ImageViews, layouts, etc.

🔧 Types of Drawable Resources:

Type	Description	Example
Bitmap Drawable	PNG, JPG, GIF images	@drawable/logo.png
Shape Drawable	XML files that define shapes like rectangles, etc.	@drawable/rounded_button.xml
State List Drawable	Changes image based on state (pressed, focused)	Buttons with different states
Layer List Drawable	Multiple drawables stacked together	Background with shadow
Vector Drawable	Scalable images in XML (small size, sharp quality)	@drawable/ic_home.xml
Nine-patch Drawable	Stretchable image with .9.png extension	Stretchable button background

✓ Where Are They Stored?

📁 Inside the res/drawable/ folder:

- res/drawable/ — default
- res/drawable-hdpi/, drawable-mdpi/, drawable-xhdpi/ — for different screen sizes

✓ Using Drawable in Layout (XML):

```
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/logo" />
```

✓ Creating a Shape Drawable Example (XML):

res/drawable/rounded_button.xml

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#FF6200EE"/>
    <corners android:radius="10dp"/>
</shape>
```

🏁 Conclusion:

Drawable Resources are used to add images and graphical elements in Android apps. They make the UI look attractive, support different screen sizes, and allow for creative visual designs using XML or image files.

2.2.3 Activity Java file

◆ What is an Activity Java File?

- An Activity Java file is the main code file that controls the screen (UI) of an Android app.
- Each screen in an app is called an Activity, and the Java file connected to that screen is called the Activity Java file.

💡 Think of it like the brain behind the screen. It tells what should happen when a user clicks a button, opens the screen, or performs any action.

✓ Main Purpose of an Activity Java File:

- Controls what happens on the screen
- Handles user interactions (clicks, inputs, etc.)
- Connects the UI design (XML) with the backend code (Java/Kotlin)

✓ Basic Structure of an Activity Java File:

```
package com.example.myapplication;

import android.os.Bundle;
import android.app.Activity;
import android.widget.TextView;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Set the layout (XML) for this activity
        setContentView(R.layout.activity_main);

        // Example: Showing text on screen
        TextView textView = findViewById(R.id.myTextView);
        textView.setText("Hello, Android!");
    }
}
```

✓ Important Points:

Part	Description
extends Activity or AppCompatActivity	Means this class is an Activity
onCreate()	This method runs when the Activity is started
setContentView()	Connects XML layout to Java code
findViewById()	Used to access UI elements like buttons, text views, etc.

✓ How it Works:

1. App starts
2. Android system loads MainActivity.java
3. It runs onCreate() method
4. It sets the layout (e.g., activity_main.xml)
5. Handles user actions like clicks

Example Use-Cases:

- Button clicks
- Navigating to another screen
- Showing messages (Toast)
- Performing calculations or logic

Conclusion:

The Activity Java file is the code behind each screen in an Android app. It connects the design with logic and makes the app interactive and functional. Without it, your app screens would just be empty designs.

2.2.4 Gradle

Introduction to Gradle:

- Gradle is an automated build system used to compile, build, and manage Android projects.
- It handles the process of converting source code into an APK (Android application) file that can be installed on a device.

 Think of Gradle like a project manager that organizes and automates tasks such as building, testing, and packaging your app.

Why is Gradle Important?

- Automation: Gradle automatically handles repetitive tasks like compiling code, building APKs, and managing dependencies.
- Efficiency: It speeds up the build process by caching results of previous tasks.
- Flexibility: It can work with multiple programming languages and frameworks, not just Java or Android.

How Gradle Works in Android Projects:

1. Gradle Scripts:

- build.gradle: These are script files written in Groovy or Kotlin that contain instructions for Gradle.
- There are two types of build.gradle files:
 - Project-level build.gradle: Contains global settings like repositories, plugins, etc.
 - App-level build.gradle: Contains specific settings for your app like dependencies, SDK versions, etc.

2. Gradle Tasks:

- Gradle executes tasks like compile, test, build, clean, etc.
- These tasks can be automated and executed with a single command (e.g., gradle build).

✓ Structure of Gradle Files in Android:

1. Project-level build.gradle:

- Contains settings shared across the whole project (e.g., repositories, Gradle version).

```
buildscript {  
    repositories {  
        google()  
        jcenter()  
    }  
    dependencies {  
        classpath 'com.android.tools.build:gradle:4.1.0'  
    }  
}  
  
allprojects {  
    repositories {  
        google()  
        jcenter()  
    }  
}
```

2. App-level build.gradle:

- Defines specific settings for the app (e.g., app dependencies, SDK versions).

```
apply plugin: 'com.android.application'  
  
android {  
    compileSdkVersion 30  
    defaultConfig {  
        applicationId "com.example.myapp"  
        minSdkVersion 21  
        targetSdkVersion 30  
        versionCode 1  
        versionName "1.0"  
    }  
}  
  
dependencies {  
    implementation 'com.android.support:appcompat-v7:28.0.0'  
    implementation 'com.google.android.material:material:1.1.0'  
}
```

✓ Key Gradle Features:

1. Dependencies Management:

- Gradle allows you to manage third-party libraries and dependencies (e.g., Retrofit, Glide) easily by adding them to the dependencies section.

2. Build Types:

- You can define different build configurations like debug or release with specific settings (e.g., signing keys for release APKs).

3. Multi-module Support:

- Gradle can handle projects with multiple modules, making it easy to structure larger apps.

4. Gradle Wrapper:

- Ensures you are using a specific Gradle version, making the build process consistent across all machines.

✓ Example of Gradle Commands:

- `gradle build`: Builds the project and generates the APK.
- `gradle clean`: Removes any previously built files.
- `gradle assembleRelease`: Builds the release version of the APK.
- `gradle dependencies`: Lists all dependencies in the project.

← END Conclusion:

Gradle is an essential tool in Android development, automating and simplifying tasks like building, testing, and managing dependencies. It helps developers focus on coding while Gradle handles the background processes to ensure your app is built efficiently and correctly.

Unit3: Layout and Advanced UI Design

3.1. Layouts and its attributes - Linear, Relative, Constraint

◆ What is a Layout in Android?

- In Android, a Layout is like a container that holds and arranges UI elements (like buttons, text, images) on the screen.
- 💡 You can think of it as the blueprint for how things are placed in your app screen.
- There are different types of layouts in Android. One of the most commonly used is `LinearLayout`.

✓ What is `LinearLayout`?

`LinearLayout` is a view group that arranges its child views in a single direction — either vertically or horizontally.

- If vertical, items are placed one below the other.

- If horizontal, items are placed side by side (left to right).
- 💡 Like stacking blocks — either top to bottom or left to right.

✓ Syntax of LinearLayout (XML):

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Click Me" />
</LinearLayout>
```

✓ Important Attributes of LinearLayout:

Attribute	Meaning
android:orientation	Direction of layout - vertical or horizontal
android:layout_width	Width of the element (match_parent, wrap_content, or a specific value)
android:layout_height	Height of the element
android:layout_weight	Controls how much space an item takes relative to others
android:gravity	Aligns all items inside the layout (e.g., center, start)
android:layout_gravity	Aligns a specific item inside the parent layout
android:padding	Space inside the view (like inner margin)
android:margin	Space outside the view (space between two views)

✓ Types of LinearLayout Orientation:

◆ Vertical Layout:

Items go from top to bottom.

```
<LinearLayout android:orientation="vertical">
    <!-- Items placed one below the other -->
</LinearLayout>
```

◆ Horizontal Layout:

Items go from left to right.

```
<LinearLayout android:orientation="horizontal">
    <!-- Items placed side by side -->
</LinearLayout>
```

✓ Advantages of LinearLayout:

- Very easy to use for simple layouts.
- Useful for creating lists or rows of elements.
- Works well when you have limited UI elements.

✓ Disadvantages:

- Not good for complex UIs.
- May become slow if nested too many times (deep hierarchy).

← Conclusion:

LinearLayout is one of the most basic and important layouts in Android. It helps developers arrange elements in a straight line — either vertically or horizontally. It is simple to use and best for creating small and clean layouts.

✓ What is RelativeLayout?

RelativeLayout is a type of layout where you can place UI elements (views) in relation to each other or to the parent layout.

💡 Think of it like putting sticky notes on a board — you can place one below, next to, centered, or aligned with another note.

✓ How it Works:

- You can position one view relative to:
 - Another view (e.g., to the right of a TextView)
 - The parent (e.g., align to the top, center, bottom)

✓ Basic Example of RelativeLayout:

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/text1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```



```
android:text="Hello" />
```

```
<Button
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Click Me"
```

```
    android:layout_below="@id/text1"
```

```
    android:layout_marginTop="20dp" />
```

```
</RelativeLayout>
```

In this example:

- The TextView is placed at the top.
- The Button is placed below the TextView using `android:layout_below="@id/text1"`.

✓ Common Attributes of RelativeLayout:

Attribute	What It Does
<code>android:layout_below="@id/viewID"</code>	Places the view below another view
<code>android:layout_above="@id/viewID"</code>	Places the view above another view
<code>android:layout_toRightOf="@id/viewID"</code>	Places the view to the right of another
<code>android:layout_toLeftOf="@id/viewID"</code>	Places the view to the left of another
<code>android:layout_alignParentTop="true"</code>	Aligns view to the top of parent
<code>android:layout_alignParentBottom="true"</code>	Aligns view to the bottom of parent
<code>android:layout_centerHorizontal="true"</code>	Centers view horizontally in parent
<code>android:layout_centerVertical="true"</code>	Centers view vertically in parent
<code>android:layout_centerInParent="true"</code>	Centers view both ways in parent
<code>android:id="@+id/xyz"</code>	Gives a unique ID to refer to this view in other rules

✓ Advantages of RelativeLayout:

- Very flexible – You can arrange views almost any way you want.
- Good for complex UIs with many elements.
- Helps avoid deep nesting of layouts, which improves performance.

✓ Disadvantages:

- Can be harder to understand and maintain, especially for beginners.
- Layout rules must be carefully written to avoid confusion.

✓ Use Cases:

- Login forms (username, password fields)
- Custom screens where elements must be placed relative to others

- Centering elements in the screen

🔙 Conclusion:

RelativeLayout allows you to place views relative to each other or to the parent. It's powerful and flexible, perfect for building more advanced screen layouts in Android apps. Learning its attributes gives you more control over the user interface.

◆ What is ConstraintLayout?

ConstraintLayout is a powerful and flexible layout in Android that allows you to position UI elements by setting constraints (rules) relative to:

- Other views
- Parent layout
- Guidelines

💡 Think of it like placing items on a grid and using strings (constraints) to tie them to each other or to the edges of the screen.

✓ Why Use ConstraintLayout?

- Helps create complex layouts using a single layout file (no need to nest multiple layouts).
- Improves performance by reducing the depth of the view hierarchy.
- Allows drag-and-drop designing in Android Studio.

✓ Basic Example of ConstraintLayout:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, World!"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```

✦ This example places the TextView at the top center of the screen by connecting it to the top, start, and end of the parent layout.

✓ Important Attributes of ConstraintLayout:

Attribute	What It Does
<code>app:layout_constraintTop_toTopOf="..."</code>	Attach the top of the view to another view or the parent
<code>app:layout_constraintBottom_toBottomOf="..."</code>	Attach the bottom of the view
<code>app:layout_constraintStart_toStartOf="..."</code>	Attach the left/start side
<code>app:layout_constraintEnd_toEndOf="..."</code>	Attach the right/end side
<code>app:layout_constraintLeft_toLeftOf="..."</code>	Same as Start (for older devices)
<code>app:layout_constraintRight_toRightOf="..."</code>	Same as End (for older devices)
<code>app:layout_constraintHorizontal_bias="0.5"</code>	Controls horizontal positioning (0 = left, 1 = right, 0.5 = center)
<code>app:layout_constraintVertical_bias="0.5"</code>	Controls vertical positioning
<code>app:layout_constraintWidth_percent="0.5"</code>	Set width as a percentage of the parent
<code>app:layout_constraintHeight_percent="0.3"</code>	Set height as a percentage

✓ Advantages of ConstraintLayout:

- ✓ Less nesting of layouts → better performance
- ✓ Create complex UIs in one layout
- ✓ Easy to design visually in Android Studio
- ✓ Supports percentage-based width and height
- ✓ Works well on different screen sizes

✓ Disadvantages:

- ✗ Can be confusing at first due to many constraints
- ✗ Writing manually in XML can be complex (but drag-and-drop helps)

✓ Use Cases:

- Splash screens, login screens, and dashboards
- Any screen with complex alignment of views
- Responsive UIs that look good on all devices

← END Conclusion:

ConstraintLayout is the most modern and efficient layout in Android. It lets you build clean, flexible, and high-performance UIs by setting rules (constraints) between elements. Once you get used to it, it makes layout design much easier and more powerful.

3.2. Scrollview

◆ What is ScrollView?

- In Android, ScrollView is a layout that allows the user to scroll through content

vertically when it does not fit on the screen.

💡 Imagine reading a long page or form – when the screen is full, you scroll down to see more. That's what ScrollView does.

✓ Why Do We Use ScrollView?

When you have more content than can be shown on one screen (like a long form, big text, or multiple images), you use ScrollView so the user can scroll and view all the content.

✓ Basic Example of ScrollView:

```
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Your scrollable content here -->
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="This is a long paragraph..." />

        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Click Me" />

        <!-- Add more views -->

    </LinearLayout>
</ScrollView>
```

✦ Important Rule: ScrollView can only have one direct child layout (like LinearLayout or ConstraintLayout), and that child can contain multiple views.

✓ Types of ScrollView:

Type	Description
ScrollView	For vertical scrolling
HorizontalScrollView	For left-right (horizontal) scrolling

✓ Important Attributes of ScrollView:

Attribute	Description
android:fillViewport="true"	Forces the content to fill the screen (even if it's smaller)
android:scrollbars="vertical"	Shows vertical scrollbar (optional)
android:layout_width	Width of the ScrollView (usually match_parent)
android:layout_height	Height of the ScrollView (usually match_parent)

✓ Advantages of ScrollView:

- ✓ Makes your UI accessible even if it's long
- ✓ Easy to use and understand
- ✓ Helps show forms, images, long text, etc.

✓ Disadvantages:

- ✗ Only scrolls in one direction (vertical or horizontal)
- ✗ Should not be used with RecyclerView or ListView (as they handle their own scrolling)

✓ When to Use ScrollView:

- Long registration or login forms
- Pages with large amounts of text
- Layouts that must fit different screen sizes

← END Conclusion:

ScrollView is a simple layout in Android used to make your screen scrollable. It's best when you have long content that doesn't fit on a single screen. Just wrap your main layout inside a ScrollView, and Android will take care of the scrolling!

3.3. Webview

◆ What is WebView?

- WebView is a special view in Android that is used to display web pages (websites) inside your app — without opening a browser.

💡 Think of it like putting a mini Chrome browser inside your app.

✓ Why Use WebView?

- To show a website inside the app.
- To display online content like terms and conditions, blogs, etc.
- To create hybrid apps (half website, half Android app).

✓ Basic Example of WebView:

XML Layout File:

```
<WebView
    android:id="@+id/myWebView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Java/Kotlin Code:

```
WebView webView = findViewById(R.id.myWebView);
webView.getSettings().setJavaScriptEnabled(true); // enable JavaScript if needed
webView.loadUrl("https://www.google.com"); // load website
```

✓ Important Attributes (in XML):

Attribute	Description
android:id	Unique ID to use in code
android:layout_width	Width of the WebView
android:layout_height	Height of the WebView

✓ Important Methods (in Java/Kotlin):

Method	What It Does
loadUrl("https://example.com")	Loads the given website URL
getSettings().setJavaScriptEnabled(true)	Enables JavaScript
goBack()	Goes back to the previous page
reload()	Refreshes the web page
canGoBack()	Checks if there is a back history

✓ Advantages of WebView:

- ✓ Easy to show web content inside your app
- ✓ Saves user from switching to browser
- ✓ Supports HTML, CSS, JavaScript

✓ Disadvantages:

- ✗ May be slower than native Android views
- ✗ Not secure if you load unknown/untrusted URLs
- ✗ Needs internet connection for online pages

✓ Use Cases of WebView:

- Showing your company website in the app
- Displaying online help pages
- Opening YouTube videos, news articles, or blog posts
- Loading HTML files stored in the assets folder

🏠 Conclusion:

WebView is a powerful Android component that lets you open and view websites inside your app. It's useful for showing online content without leaving the app. Just use it wisely and make sure the websites you load are safe.

3.4. Seekbar, Rating bar

✓ SeekBar in Android

◆ What is SeekBar?

- A SeekBar is a UI (User Interface) element in Android that lets the user select a value by sliding a thumb (circle) along a horizontal line.

💡 Example: Volume control, brightness level, music progress bar — all use SeekBar.

✓ Why Use SeekBar?

SeekBar is used when you want the user to pick a value from a range, like 0 to 100.

✓ How SeekBar Looks:

- A horizontal line (track)
- A movable circle (thumb)
- You slide the thumb left or right to change the value.

✓ Basic Example of SeekBar:

XML Layout File:

```
<SeekBar
    android:id="@+id/mySeekBar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:max="100"
    android:progress="50" />
```

Java/Kotlin Code:

```
SeekBar seekBar = findViewById(R.id.mySeekBar);
```

```
seekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override
    public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser)
    {
        // Called when the thumb is moved
        Toast.makeText(getApplicationContext(), "Progress: " + progress,
        Toast.LENGTH_SHORT).show();
    }
})
```

```

@Override
public void onStartTrackingTouch(SeekBar seekBar) {
    // Called when user starts moving the thumb
}

@Override
public void onStopTrackingTouch(SeekBar seekBar) {
    // Called when user stops moving the thumb
}
});

```

✓ Important Attributes in XML:

Attribute	Description
android:max="100"	Sets the maximum value
android:progress="50"	Sets the current (default) value
android:thumb="@drawable/thumb_image"	Customizes the thumb image
android:layout_width / height	Controls size of SeekBar

✓ Common Use Cases:

- 📶 Volume control
- 💡 Brightness control
- ▶▶ Media progress (music, video)
- 🎮 Game speed/level adjuster

✓ Advantages of SeekBar:

- ✓ Simple and user-friendly
- ✓ Easy to implement
- ✓ Good for adjusting values with quick feedback

✓ Disadvantages:

- ✗ Not suitable for selecting exact values (like typing age or PIN)
- ✗ Can be hard to control with precision

🏠 Conclusion:

SeekBar is a helpful Android widget that lets users select a value by sliding. It's widely used in media apps, settings, and games. With just a few lines of code, you can make your app more interactive and dynamic.

✓ RatingBar in Android

◆ What is RatingBar?

- RatingBar is a UI (User Interface) component in Android that allows users to rate something using stars (like 1 to 5 stars).

💡 Example: You give 4 stars to a movie or product — that's a RatingBar in action!

✓ Why Use RatingBar?

- To collect user ratings for movies, products, apps, services, etc.
- To show the average rating of something.
- It gives a simple visual way to express a score.

✓ Basic Example of RatingBar in XML:

```
<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:stepSize="1.0"
    android:rating="3"
    style="@style/Widget.AppCompat.RatingBar.Small"/>
```

This creates a 5-star RatingBar, starting at 3 stars.

✓ Java/Kotlin Code Example:

```
RatingBar ratingBar = findViewById(R.id.ratingBar);

ratingBar.setOnRatingBarChangeListener(new
RatingBar.OnRatingBarChangeListener() {
    @Override
    public void onRatingChanged(RatingBar ratingBar, float rating, boolean
fromUser) {
        Toast.makeText(getApplicationContext(), "Rating: " + rating,
        Toast.LENGTH_SHORT).show();
    }
});
```

✦ This code shows the rating in a toast message when the user changes it.

✓ Important Attributes:

Attribute	Meaning
android:numStars="5"	Total number of stars (e.g. 5)
android:rating="3"	Default rating (e.g. 3 stars)
android:stepSize="0.5"	Allows half-star ratings

android:isIndicator="true"	Makes it read-only (user can't change it)
android:layout_width/height	Size of the RatingBar

✓ Advantages of RatingBar:

- ✓ Easy to use and understand
- ✓ Visually shows the rating
- ✓ Can get feedback directly from users

✓ Disadvantages:

- ✗ May not be accurate feedback if users randomly click
- ✗ Needs additional logic to store or process rating

✓ Use Cases of RatingBar:

- Movie or product reviews
- Rating an app feature
- Giving feedback in forms
- Rating after delivery or service



Conclusion:

RatingBar is a simple and useful Android UI tool for taking or displaying ratings using stars. It improves user interaction and helps collect feedback in an attractive way.

3.5. RecyclerView

◆ What is RecyclerView?

- RecyclerView is a view widget in Android used to display large sets of data in a scrollable list or grid.
- It is more flexible and efficient than the older ListView, especially for displaying large amounts of data, because it recycles views to improve performance.

💡 Think of it like a supercharged list that not only shows items but also reuses the items that are no longer visible, making it faster and smoother.

✓ Why Use RecyclerView?

- Efficient Memory Usage: RecyclerView reuses the views that scroll off the screen, which saves memory.
- Smooth Scrolling: Handles large datasets smoothly.
- Flexible Layouts: RecyclerView can be customized for different layouts (linear, grid, staggered, etc.).
- Item Animations: RecyclerView supports adding animations when items are added, removed, or moved.

✓ Basic Components of RecyclerView:

1. RecyclerView: This is the view container that displays the list/grid.
2. ViewHolder: Holds references to the views inside the individual items of the list.
3. Adapter: Binds the data to the views inside the RecyclerView. It tells what data should be shown for each item.
4. LayoutManager: Defines the layout and structure of how items are arranged (e.g., a vertical list, grid, etc.).

✓ How RecyclerView Works:

1. Adapter: It connects the data with the RecyclerView. It creates ViewHolder objects for each item in the list.
2. ViewHolder: It caches the view elements (like TextViews, ImageViews) for each item, so they don't need to be created again and again.
3. LayoutManager: It arranges the ViewHolder objects in a certain layout (linear, grid, etc.).
4. RecyclerView: It uses the ViewHolder, Adapter, and LayoutManager to display and manage the items in the list.

✓ Basic Example of RecyclerView:

1. Add RecyclerView to XML Layout:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

2. Create a ViewHolder Class:

```
public class ItemViewHolder extends RecyclerView.ViewHolder {
    TextView textView;

    public ItemViewHolder(View itemView) {
        super(itemView);
        textView = itemView.findViewById(R.id.textView);
    }
}
```

3. Create an Adapter Class:

```
public class ItemAdapter extends RecyclerView.Adapter<ItemViewHolder> {
    private List<String> items;

    public ItemAdapter(List<String> items) {
        this.items = items;
    }

    @Override
    public ItemViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view =
        LayoutInflater.from(parent.getContext()).inflate(R.layout.item_layout, parent, false);
        return new ItemViewHolder(view);
    }
}
```

```

    }

    @Override
    public void onBindViewHolder(ItemViewHolder holder, int position) {
        holder.textView.setText(items.get(position));
    }

    @Override
    public int getItemCount() {
        return items.size();
    }
}

```

4. Set up RecyclerView in Activity:

```

RecyclerView recyclerView = findViewById(R.id.recyclerView);
recyclerView.setLayoutManager(new LinearLayoutManager(this)); // You can change it
to GridLayout or StaggeredLayout
recyclerView.setAdapter(new ItemAdapter(myItemList)); // Provide the data

```

✓ LayoutManager Types:

1. LinearLayoutManager: Displays items in a vertical or horizontal list.
2. GridLayoutManager: Displays items in a grid (multiple columns/rows).
3. StaggeredGridLayoutManager: Displays items in a staggered grid (useful for images or variable-height items).

✓ Advantages of RecyclerView over ListView:

Feature	RecyclerView	ListView
Reusability	Reuses items that are no longer visible	Does not reuse items, creates a new view every time
Performance	More efficient with large datasets	Less efficient for large data
Layout Flexibility	Supports linear, grid, and staggered layouts	Limited to vertical lists
Animations	Built-in support for item animations	Does not support built-in animations

🏠 Conclusion:

RecyclerView is a powerful tool in Android that allows you to efficiently display large sets of data in a list or grid. By recycling views and using flexible layouts, it enhances both the performance and the user experience of your app.

3.6. Alert Dialog

◆ What is an AlertDialog?

- An AlertDialog is a pop-up window that is used to show messages or ask the user for a decision.
- It is used to alert the user, ask for confirmation, or show important information like warnings or errors.

💡 Think of it like a pop-up notification in your app that grabs the user's attention and asks them to either confirm, cancel, or just read a message.

✓ When to Use AlertDialog?

- When you need to ask the user for confirmation (e.g., "Do you want to delete this file?").
- To show a simple message to the user (e.g., an error or success message).
- To give the user options to choose from (e.g., multiple buttons).

✓ Components of an AlertDialog:

- Title: A heading or title for the dialog box (optional).
- Message: The main text or content shown in the dialog.
- Buttons: These buttons perform actions like OK, Cancel, or custom actions.
 - Positive Button: Typically used for a confirmation action (e.g., "Yes").
 - Negative Button: Typically used for a dismissal action (e.g., "No").
 - Neutral Button: Used for neutral actions (e.g., "Later").

✓ Creating and Using an AlertDialog:

Here's an example of how to create and display an AlertDialog:

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Confirm Deletion")
    .setMessage("Are you sure you want to delete this file?")
    .setPositiveButton("Yes", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            // Action when Yes is clicked
            deleteFile();
        }
    })
    .setNegativeButton("No", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            // Action when No is clicked
            dialog.dismiss();
        }
    })
    .show(); // Display the dialog
```

✓ Explanation of Code:

- AlertDialog.Builder: This is used to build the dialog.
- setTitle(): Sets the title of the dialog.

- setMessage(): Sets the message in the dialog.
- setPositiveButton(): Defines the action when the "Yes" button is clicked.
- setNegativeButton(): Defines the action when the "No" button is clicked.
- show(): Displays the dialog on the screen.

✓ Types of AlertDialogs:

1. Simple Message Dialog: Shows a message with only OK or Close buttons.
 - Example: Informing the user of an error.
2. Confirmation Dialog: Asks the user for a decision (e.g., "Yes" or "No").
 - Example: Deleting a file.
3. Choice Dialog: Displays options like a list or radio buttons.
 - Example: Letting the user select from multiple options (e.g., Choose a color).

✓ Example of a List Choice Dialog:

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Choose a Fruit")
    .setItems(new String[] {"Apple", "Banana", "Orange"}, new
DialogInterface.OnClickListener() {
    @Override
    public void onClick(DialogInterface dialog, int which) {
        // Action based on which item is selected
        String selectedFruit = (which == 0) ? "Apple" : (which == 1) ? "Banana"
: "Orange";
        Toast.makeText(getApplicationContext(), "You selected: " +
selectedFruit, Toast.LENGTH_SHORT).show();
    }
})
.show();
```

← END Conclusion:

An AlertDialog is a simple, interactive pop-up that helps communicate with the user and allows them to make decisions or acknowledge messages. It is essential for handling confirmations, warnings, and simple user interactions in Android apps.

Unit 4: Navigating across Application

4.1. Intent and Intent Filters

✓ Intent in Android

◆ What is Intent?

- In Android, an Intent is like a message or request used to start an activity, service, or send data from one screen to another.

💡 Think of Intent as a messenger that helps you move from one screen to another, or share information between apps.

✓ Why Use Intent?

- To start a new activity (go to a new screen)
- To pass data between activities
- To open other apps (like Camera, Browser, etc.)
- To start background services

✓ Types of Intent:

Type-Description-Example

Explicit Intent-Used to start a specific activity within your app-Open ProfileActivity from MainActivity

Implicit Intent-Used to perform an action using another app-Open browser, send email, share image

✓ Example of Explicit Intent:

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

✦ This opens SecondActivity from MainActivity.

✓ Passing Data with Intent:

```
Intent intent = new Intent(this, SecondActivity.class);
intent.putExtra("username", "Rishi");
startActivity(intent);
```

In SecondActivity:

```
String name = getIntent().getStringExtra("username");
```

✦ This sends data (like a username) to another screen.

✓ Example of Implicit Intent:

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.google.com"));
startActivity(intent);
```

✦ This opens Google in a browser app.

✓ Common Intent Actions:

Action	Use
ACTION_VIEW	View something (e.g., a webpage or contact)

ACTION_SEND Send data (e.g., share text or image)
ACTION_DIAL Open the dialer with a number
ACTION_CAMERA Open the camera app

✓ Advantages of Intent:

- ✓ Easy to move between screens
- ✓ Helps apps communicate with each other
- ✓ Allows data sharing
- ✓ Very flexible and powerful

✓ Disadvantages:

- ✗ Can cause errors if the intent is not handled properly
- ✗ Security concerns if sensitive data is sent without protection

✓ Use Cases of Intent:

- Navigating between login and home screen
- Opening the camera or gallery
- Sending emails or sharing text/images
- Starting background tasks (like music or downloading)

← END Conclusion:

Intent is one of the most important features in Android. It helps you navigate, send data, and communicate between apps easily. Understanding Intents is key to building interactive Android apps.

✓ Intent Filters in Android

◆ What is an Intent Filter?

- An Intent Filter is a part of the AndroidManifest.xml file that tells the Android system what your app can do or what kind of data it can handle.

💡 It helps Android decide which activity to open when the user performs an action like opening a link, viewing an image, or sharing data.

✓ Why Use Intent Filters?

- To launch your activity when a user clicks a link or opens a file.
- To handle specific actions, like sending or receiving SMS, opening a webpage, etc.
- To make your app available for certain system actions (like setting your app as the default browser).

✓ Basic Example of an Intent Filter:


```

<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

```

◆ This is the default intent filter for the main launcher activity (app icon click).

✓ Another Example: Open App on Web Link

```

<activity android:name=".WebActivity">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https"
              android:host="www.example.com" />
    </intent-filter>
</activity>

```

◆ This tells Android: "If the user opens https://www.example.com, open WebActivity."

✓ Main Parts of an Intent Filter:

Tag	Purpose
<action>	What kind of action it handles (e.g., VIEW, SEND, MAIN)
<category>	Additional info about the action (e.g., LAUNCHER, DEFAULT)
<data>	What kind of data the app handles (e.g., URLs, file types, MIME types)

✓ Advantages of Intent Filters:

- ✓ Helps your app respond to system events
- ✓ Makes your app available to handle shared content
- ✓ Allows deep linking to your app from websites

✓ Disadvantages:

- ✗ Can make the app vulnerable if not used carefully
- ✗ Can conflict with other apps if too broad

✓ Use Cases:

- Opening your app from a web link
- Receiving a shared image from the gallery
- Setting your app as the default SMS app
- Receiving broadcasts from the system

← END Conclusion:

Intent Filters are like invitations that tell Android, "Hey, my app can handle this action!" They make your app more interactive and help it connect with the system and other apps. Proper use of intent filters improves the user experience and adds powerful functionality.

4.2. Menus - context, pop-up, options

✓ Context Menu in Android

◆ What is a Context Menu?

- A Context Menu is a menu that appears when the user long presses (or right-clicks on some devices) on an item, giving the user options to perform actions related to that item.

💡 Think of it like right-clicking on a file or an image on a computer to show actions like "Copy," "Delete," or "Rename."

✓ Why Use Context Menu?

- To give the user actions related to specific items (like a list or an image).
- To keep the UI clean by showing options only when the user needs them.
- To allow for more interactivity without taking up space in the app's main layout.

✓ Basic Example of a Context Menu:

1. Creating a Context Menu in XML:

- You don't need XML for the context menu itself; the actions are handled in Java code.

2. Code to Create and Handle Context Menu:

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
ContextMenu.ContextMenuInfo menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    // Inflate the context menu
    getMenuInflater().inflate(R.menu.context_menu, menu);
}
```

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.action_delete:
            // Code for deleting the item
    }
}
```

```

        return true;
    case R.id.action_share:
        // Code for sharing the item
        return true;
    default:
        return super.onContextItemSelected(item);
    }
}

```

In this example:

- onCreateContextMenu() is used to create the context menu when a user long presses on an item.
- onContextItemSelected() is used to handle the actions chosen from the menu (e.g., delete or share).

Menu XML File (res/menu/context_menu.xml):

```

<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/action_delete"
        android:title="Delete" />
    <item
        android:id="@+id/action_share"
        android:title="Share" />
</menu>

```

This defines the options in the context menu: Delete and Share.

✓ How to Show the Context Menu:

- To trigger the context menu, you need to register the view (like a button or list item) to show the context menu when the user long presses it.

Example:

```

View itemView = findViewById(R.id.myItem);
itemView.setOnCreateContextMenuListener(this);

```

✓ Key Methods to Work with Context Menu:

Method	Description
onCreateContextMenu()	Used to create the context menu
onContextItemSelected()	Handles user actions (like delete, share)
registerForContextMenu(View view)	Registers the view to show the context menu

✓ Advantages of Context Menu:

- ✓ Efficient: Shows actions only when needed (long press).
- ✓ Clean UI: Keeps the interface simple without showing unnecessary options.
- ✓ User-Friendly: Makes it easy for users to interact with specific items.

✓ Disadvantages:

- ✗ Can be confusing for new users if not used properly.
- ✗ May not be suitable for all actions, especially if the options are always visible.
- ✗ Takes up extra screen space on devices with small screens if not used carefully.

✓ Use Cases of Context Menu:

- Deleting an item from a list or gallery (e.g., "Delete" option).
- Editing an item in a list or contact (e.g., "Edit" option).
- Providing actions for single items in a list (like a to-do app or contact list).

←_{END} Conclusion:

A Context Menu in Android is a menu that appears when the user long-presses on an item, offering actions related to that item. It helps keep your app's interface clean, user-friendly, and provides an efficient way to interact with specific items.

✓ Popup Menu in Android

◆ What is a Popup Menu?

- A Popup Menu is a menu that appears when the user clicks on a view (like a button or an item), and it provides a list of actions or options for the user to choose from. It is similar to a context menu, but it appears on top of the UI, usually when triggered by a click or tap event.

💡 Example: Imagine clicking on the 3-dot menu icon in many apps to show a list of options like "Refresh," "Settings," or "Help."

✓ Why Use Popup Menu?

- To give users quick access to actions for a specific item.
- To keep the app's UI clean and not cluttered.
- To display options without leaving the current screen (like a dropdown).

✓ Basic Example of Popup Menu:

XML Layout File (activity_main.xml):

```
<Button
    android:id="@+id/myButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
android:text="Show Popup Menu"/>
```

Java/Kotlin Code:

```
Button myButton = findViewById(R.id.myButton);
```

```
myButton.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        PopupMenu popupMenu = new PopupMenu(MainActivity.this, myButton);  
        MenuInflater inflater = popupMenu.getMenuInflater();  
        inflater.inflate(R.menu.popup_menu, popupMenu.getMenu());  
  
        // Show the menu  
        popupMenu.show();  
  
        // Handle item click  
        popupMenu.setOnMenuItemClickListener(new  
PopupMenu.OnMenuItemClickListener() {  
            @Override  
            public boolean onMenuItemClick(MenuItem item) {  
                switch (item.getItemId()) {  
                    case R.id.action_settings:  
                        // Handle settings action  
                        return true;  
                    case R.id.action_help:  
                        // Handle help action  
                        return true;  
                    default:  
                        return false;  
                }  
            }  
        });  
    }  
});
```

Menu XML File (res/menu/popup_menu.xml):

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">  
    <item  
        android:id="@+id/action_settings"  
        android:title="Settings" />  
    <item
```

```
        android:id="@+id/action_help"
        android:title="Help" />
    </menu>
```

In this example:

- A button triggers the popup menu.
- The menu shows "Settings" and "Help" options.
- When an option is selected, the code handles the click event and performs an action.

✔ How Popup Menu Works:

Step 1: You create a Button or any other view in your layout.

Step 2: When the user clicks on the view, you create a PopupMenu object in the Java/Kotlin code.

Step 3: You define the menu items using a Menu XML file.

Step 4: You use the show() method to display the menu when the view is clicked.

Step 5: You handle the item clicks to perform specific actions.

✔ Important Methods of PopupMenu:

Method	Description
popupMenu.show()	Shows the popup menu.
popupMenu.getMenuInflater().inflate()	Inflates the menu items from a menu XML file.
popupMenu.setOnMenuItemClickListener()	Sets the listener to handle item clicks from the menu.

✔ Advantages of Popup Menu:

- ✔ Simple to implement and use.
- ✔ Takes up little space on the screen, keeping the UI clean.
- ✔ Flexible: You can show a list of actions without leaving the current screen.
- ✔ Fast access to options for the user.

✔ Disadvantages:

- ✗ May be misleading if not used properly (e.g., if the menu options are not intuitive).
- ✗ Limited in terms of advanced actions compared to context menus.
- ✗ Not suitable for more complex menus with many options.

✔ Use Cases of Popup Menu:

- Settings and Help options in the app.
- Sharing options (e.g., "Share via Email," "Share via Social Media").
- Navigation actions (e.g., "Go Home," "Go to Profile").

- Actions for a specific item (e.g., a list item or a contact).

🔙 Conclusion:

A Popup Menu in Android is a small, temporary menu that appears when a user interacts with a view (like a button). It provides a quick way for users to select actions without cluttering the screen. It's easy to implement and works well for displaying a few options.

✔ Options Menu in Android

◆ What is an Options Menu?

- An Options Menu is a menu that appears when the user taps on the menu button (usually represented by three dots in the top-right corner of the app) or presses the physical menu button (on older Android devices). This menu typically contains actions that the user can perform globally in the app, such as settings, search, or help.

💡 Example: In many apps, when you tap the 3-dot menu icon in the top-right corner, options like "Settings," "Log Out," or "Help" appear — that's the Options Menu.

✔ Why Use Options Menu?

- To give users global actions that are available across the app.
- To provide an organized place for actions that are not related to a specific screen.
- To keep the interface clean and uncluttered, while still providing useful options.

✔ Basic Example of an Options Menu:

1. Code to Create an Options Menu:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu items from a menu XML file
    getMenuInflater().inflate(R.menu.options_menu, menu);
    return true; // Return true to display the menu
}
```

2. Code to Handle Item Selection:

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.action_settings:
            // Handle the Settings option
            return true;
        case R.id.action_help:
            // Handle the Help option
            return true;
    }
}
```

```

        default:
            return super.onOptionsItemSelected(item);
    }
}

```

Menu XML File (res/menu/options_menu.xml):

```

<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/action_settings"
        android:title="Settings"
        android:icon="@drawable/ic_settings"
        android:showAsAction="ifRoom" />
    <item
        android:id="@+id/action_help"
        android:title="Help"
        android:icon="@drawable/ic_help"
        android:showAsAction="ifRoom" />
</menu>

```

In this example:

- onCreateOptionsMenu() inflates the Options Menu from an XML file.
- onOptionsItemSelected() handles the item selection (e.g., when the user taps Settings or Help).
- The menu items are defined in options_menu.xml.

✓ Key Methods in Options Menu:

Method	Description
onCreateOptionsMenu(Menu menu)	Called when the options menu is being created. Use this to add items to the menu.
onOptionsItemSelected()	Handles the item selection from the options menu.
getMenuInflater().inflate()	Used to inflate the menu from an XML file.

✓ Important Attributes for Menu Items:

Attribute	Meaning
android:id	A unique identifier for the menu item.
android:title	The text that will be displayed for the item.
android:icon	The icon to display next to the item.
android:showAsAction	Controls when the item appears in the action bar (options: never, ifRoom, always).

✓ Advantages of Options Menu:

- ✓ Organizes options in a compact way, making the app clean.
- ✓ Accessible from anywhere in the app through the menu button.
- ✓ Allows for actionable items like search, settings, and more without cluttering the main UI.

✓ Disadvantages:

- ✗ The options menu may not be intuitive if the user doesn't recognize the 3-dot icon.
- ✗ May not be suitable for frequently used actions, as the user has to tap the menu first.
- ✗ On smaller screens, menu items might be hidden if there is not enough space.

✓ Use Cases of Options Menu:

- Settings: Access global settings like theme or account preferences.
- Search: Provide search functionality for the entire app.
- Help: Display instructions or FAQs.
- Logout: Allow the user to log out of the app.
- Sharing: Provide options for sharing content like links or images.

🔙 Conclusion:

The Options Menu in Android is an essential UI component that provides a list of global actions for the user. It helps keep the interface clean and uncluttered while still offering important functionalities like settings, search, and help. The options menu is accessed via the menu button and is easy to implement using XML and Java/Kotlin code.

4.3. Tab layout

◆ What is TabLayout?

- TabLayout is a UI component in Android that allows users to switch between different screens or sections by tapping on tabs.

💡 Example: In WhatsApp, tabs like Chats, Status, and Calls — those are made using TabLayout.

✓ Why Use TabLayout?

- To organize content into separate sections.
- To allow easy navigation between multiple screens within the same activity.
- To give a clean and professional look to your app.

✓ How TabLayout Works

- TabLayout is usually used with ViewPager or ViewPager2. Each tab is linked to a page, and when the user taps a tab, the related page is shown.

✓ Basic Components

- TabLayout – Displays the tabs (e.g., "Home", "Profile", "Settings").
- ViewPager2 – Handles the swipe between pages (fragments).
- Fragments – Each tab can show different UI by loading a different fragment.

✓ Example of TabLayout Usage

1. Add dependencies in build.gradle:

```
implementation 'com.google.android.material:material:1.6.0'
```

2. XML Layout (activity_main.xml):

```
<com.google.android.material.tabs.TabLayout
    android:id="@+id/tabLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

```
<androidx.viewpager2.widget.ViewPager2
    android:id="@+id/viewPager"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

3. Java/Kotlin Code:

```
TabLayout tabLayout = findViewById(R.id.tabLayout);
ViewPager2 viewPager = findViewById(R.id.viewPager);

// Set up the adapter
ViewPagerAdapter adapter = new ViewPagerAdapter(this);
viewPager.setAdapter(adapter);

// Link TabLayout and ViewPager2
new TabLayoutMediator(tabLayout, viewPager,
    (tab, position) -> {
        if (position == 0) tab.setText("Home");
        else if (position == 1) tab.setText("Profile");
        else if (position == 2) tab.setText("Settings");
    }).attach();
```

✓ TabLayout Attributes (in XML):

Attribute	Description
-----------	-------------

tabMode	Controls how tabs are displayed: fixed or scrollable.
tabGravity	Alignment of tabs: fill or center.
tabIndicatorColor	Color of the underline indicator.
tabTextColor	Color of tab text.

✓ Advantages of TabLayout

- ✓ Easy to implement and use.
- ✓ Makes navigation user-friendly.
- ✓ Supports swipeable views with ViewPager2.
- ✓ Looks modern and neat.

✓ Disadvantages

- ✗ Not suitable for apps with too many tabs (use Navigation Drawer instead).
- ✗ Requires ViewPager2 and Fragments setup, which may be complex for beginners.

✓ Use Cases of TabLayout

- Social Media apps (e.g., tabs like Home, Messages, Profile).
- News apps (tabs like Top News, Sports, Technology).
- Shopping apps (tabs like Categories, Offers, Cart).

← END Conclusion

TabLayout in Android is a great way to organize content into tabs, making your app easier to use and visually attractive. It works best when combined with ViewPager2, so users can swipe between different screens or tap on the tabs.

4.4. Navigation Drawer

◆ What is a Navigation Drawer?

- A Navigation Drawer is a sliding panel that comes out from the left side of the screen. It contains a menu of options that helps users navigate to different sections of the app.

💡 Example: In apps like Gmail or YouTube, when you tap the 3 horizontal lines (≡), a side panel opens with options like Inbox, Sent, Settings, etc. That's a Navigation Drawer.

✓ Why Use a Navigation Drawer?

- To provide quick access to different parts of the app.
- To keep the main UI clean and simple.
- To allow easy navigation between app screens or features.

✓ Basic Structure of Navigation Drawer

1. DrawerLayout – A layout that contains the drawer and main content.
2. NavigationView – The actual sliding menu (left side panel).
3. Menu XML – Defines the list of items (e.g., Home, Settings, Profile).
4. Toolbar / ActionBar – Used to show the hamburger icon (≡) to open the drawer.

✓ Simple Example:

1. Add Dependencies in build.gradle:

```
implementation 'com.google.android.material:material:1.6.0'
```

2. XML Layout (activity_main.xml):

```
<androidx.drawerlayout.widget.DrawerLayout
    android:id="@+id/drawerLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Main content -->
    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@color/purple_500"
            android:title="My App" />

        <!-- Your app content here -->
    </LinearLayout>

    <!-- Navigation Drawer -->
    <com.google.android.material.navigation.NavigationView
        android:id="@+id/navigationView"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        app:menu="@menu/drawer_menu" />
```

</androidx.drawerlayout.widget.DrawerLayout>

3. Menu File (res/menu/drawer_menu.xml):

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/nav_home" android:title="Home" />
    <item android:id="@+id/nav_profile" android:title="Profile" />
    <item android:id="@+id/nav_settings" android:title="Settings" />
</menu>
```

4. Java/Kotlin Code (MainActivity.java):

```
DrawerLayout drawerLayout = findViewById(R.id.drawerLayout);
NavigationView navigationView = findViewById(R.id.navigationView);

// Handle menu item click
navigationView.setNavigationItemSelectedListener(item -> {
    int id = item.getItemId();

    if (id == R.id.nav_home) {
        // Open Home
    } else if (id == R.id.nav_profile) {
        // Open Profile
    } else if (id == R.id.nav_settings) {
        // Open Settings
    }

    drawerLayout.closeDrawers(); // Close drawer after selection
    return true;
});
```

✓ Important Attributes:

Attribute	Meaning
android:layout_gravity="start"	Positions the drawer on the left side.
app:menu	Connects the navigation drawer with a menu XML file.

✓ Advantages of Navigation Drawer:

1. ✓ Saves screen space by hiding the menu.
2. ✓ Gives a modern and professional look.
3. ✓ Easy to organize many app sections.
4. ✓ Works well with fragments and activities.

✓ Disadvantages:

1. ✗ May be hidden from users if they don't see the ☰ icon.
2. ✗ Can be complex to set up for beginners.
3. ✗ Not suitable for simple apps with few screens.

✓ Use Cases:

1. Apps with multiple features (e.g., Gmail, YouTube).
2. Apps with user profiles, settings, and categories.
3. News apps, E-commerce apps, etc.

← END Conclusion:

The Navigation Drawer is a powerful UI component that helps users easily move between different parts of an app. It slides from the left and usually includes a list of menu items like Home, Profile, and Settings. It improves user experience by making navigation smooth and clean.

4.5. Linkify

◆ What is Linkify?

1. Linkify is a feature in Android that automatically turns text into clickable links.

💡 Example: If your app displays a phone number, email address, or website URL, Linkify can make it clickable — so the user can tap it to call, send email, or open the browser.

✓ Why Use Linkify?

1. To make text more interactive.
2. To improve user experience by allowing quick actions.
3. Saves time — no need to manually create buttons or links.

✓ What Can Linkify Detect?

Linkify can detect and make links out of:

Type of Text	Action
Phone number	Opens the dialer
Email address	Opens the email app
URL (website)	Opens the browser
Map address	Opens Google Maps

✓ How to Use Linkify

1. Using Java Code:

```
TextView textView = findViewById(R.id.textView);  
textView.setText("Contact us at example@example.com or visit www.google.com");
```

Linkify.addLinks(textView, Linkify.ALL);

◆ This will automatically detect email and website and make them clickable.

2. Using XML (autoLink):

<TextView

android:id="@+id/textView"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:autoLink="all"

android:text="Call us at 1234567890 or visit https://google.com"

android:textColorLink="@android:color/holo_blue_dark"/>

◆ autoLink="all" will make phone number and URL clickable automatically.

✓ autoLink Options in XML:

Value	What It Links
none	No auto linking (default)
web	Links URLs
email	Links email addresses
phone	Links phone numbers
map	Links map addresses
all	Links all types (web, phone, email, map)

✓ Advantages of Linkify:

1. ✓ Makes the app more interactive and user-friendly.
2. ✓ Saves coding time by automatically linking text.
3. ✓ Works well with TextView in both Java/Kotlin and XML.

✓ Disadvantages:

1. ✗ Works only with TextView.
2. ✗ Doesn't allow custom styling easily.
3. ✗ Might link things you don't want (like random numbers).

✓ Use Cases of Linkify:

1. Showing contact information like phone, email, or address.
2. Displaying web URLs or social media links.
3. Creating "Terms and Conditions" with links.

🔙 Conclusion:

Linkify is a helpful Android utility that automatically detects and creates clickable links from plain text like phone numbers, emails, URLs, and map addresses. It improves user experience and makes your app more interactive without extra coding.

Unit-5: Using Shared preferences

5.1 Purpose of Shared Preferences

◆ What is Shared Preferences?

1. Shared Preferences is a way to store small amounts of data in Android. It is used to save key-value pairs like settings, user preferences, or login info.

✓ Main Purpose:

1. To store and retrieve simple data (permanently) even after the app is closed.

✓ Common Use Cases:

1. ✓ Save user login state (e.g., "Remember Me" checkbox).
2. ✓ Store settings like dark mode ON/OFF.
3. ✓ Save username, language choice, or first-time app usage info.
4. ✓ Keep track of simple app data like score, level, or flag values.

✓ How It Works:

1. Data is stored as key-value pairs.
2. Stored in XML file inside the app's internal storage.
3. Can be accessed anywhere in the app.

✓ Example in Java:

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);  
SharedPreferences.Editor editor = prefs.edit();
```

```
editor.putString("username", "Rishi");  
editor.putBoolean("isLoggedIn", true);  
editor.apply();
```

To read:

```
String name = prefs.getString("username", "Guest");  
boolean loggedIn = prefs.getBoolean("isLoggedIn", false);
```

✓ Summary:

Feature	Purpose
Simple storage	Stores key-value pairs
Data type	Strings, booleans, ints, floats
Use	User settings, preferences, small data
Lifetime	Until app is uninstalled or data is cleared

←
END Conclusion:

Shared Preferences helps Android apps to remember user data like settings or login info, even after the app is closed. It's best for small, simple data and is easy to use.

5.2 Shared Preference Modes

✓ What are Modes in SharedPreferences?

1. Modes are used to control the access level and how data is saved.
2. They are passed as the second parameter when calling:
`getSharedPreferences("MyPrefs", MODE);`

✓ Types of Shared Preference Modes:

◆ **MODE_PRIVATE** (Most Common – Default)

1. Only your app can access the data.
2. Data is private and secure.
3. Used most of the time.
4. `getSharedPreferences("MyPrefs", MODE_PRIVATE);`

✓ Best for: Storing user settings, login status, etc.

◆ **MODE_WORLD_READABLE** (Deprecated)

1. Allows other apps to read your preferences file.
2. Not secure, so it's no longer recommended.
3. Deprecated from Android 4.2 (API 17) onwards.
4. `getSharedPreferences("MyPrefs", MODE_WORLD_READABLE);`

✗ Avoid using it.

◆ **MODE_WORLD_WRITEABLE** (Deprecated)

1. Allows other apps to write (change) your preferences.
2. Very unsafe, so also deprecated.
3. `getSharedPreferences("MyPrefs", MODE_WORLD_WRITEABLE);`

✗ Avoid using it.

◆ **MODE_MULTI_PROCESS** (Deprecated)

1. Used when multiple processes of the same app want to access SharedPreferences at the same time.
2. Deprecated because of data inconsistency issues.
3. `getSharedPreferences("MyPrefs", MODE_MULTI_PROCESS);`

✗ Not recommended in newer Android versions.

◆ **MODE_APPEND** (Used with File I/O)

1. Not for SharedPreferences directly.
2. Used when working with files, not key-value preferences.

✓ Which Mode to Use?

✓ Always use MODE_PRIVATE – it's safe, secure, and supported.

✓ Example Code:

```
SharedPreferences prefs = getSharedPreferences("UserPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = prefs.edit();
```

```
editor.putString("username", "rishi");
editor.putBoolean("isLoggedIn", true);
editor.apply();
```

✓ Summary Table:

Mode	Access Level	Status
MODE_PRIVATE	Only by your app	✓ Recommended
MODE_WORLD_READABLE	Any app can read	✗ Deprecated
MODE_WORLD_WRITEABLE	Any app can write	✗ Deprecated
MODE_MULTI_PROCESS	Multiple processes	✗ Deprecated
MODE_APPEND	File append (not prefs)	✓ File use only

← Conclusion:

Shared Preference Modes control who can access your saved data.

Among all modes, MODE_PRIVATE is the safest and most used in modern Android development.

5.3 Writing to shared Preferences

◆ What Does "Writing to Shared Preferences" Mean?

It means:

➡ Saving data (as key-value pairs) in the app using SharedPreferences.

✓ Steps to Write to Shared Preferences

◆ Step 1: Create or Get SharedPreferences

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
```

1. "MyPrefs" = file name
2. MODE_PRIVATE = only your app can access the data

◆ Step 2: Start Editing

```
SharedPreferences.Editor editor = prefs.edit();
```

◆ Step 3: Put (save) Data

```
editor.putString("username", "rishi");  
editor.putBoolean("isLoggedIn", true);  
editor.putInt("score", 90);
```

◆ Step 4: Save the Changes

```
editor.apply(); // or editor.commit();
```

1. ✓ `apply()` is asynchronous (faster, recommended).
2. ✓ `commit()` is synchronous (returns true/false if saved).

✓ Full Example Code (Java):

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);  
SharedPreferences.Editor editor = prefs.edit();
```

```
editor.putString("username", "Rishi");  
editor.putBoolean("isLoggedIn", true);  
editor.apply(); // Save data
```

✓ Where is it stored?

1. Data is stored in a hidden XML file in the app's internal storage.
2. File name: "MyPrefs.xml" (if you used "MyPrefs")

✓ Use Cases:

Use Case	Example
Remember login	Save isLoggedIn = true
Save user name	Save username = "Rishi"
Theme settings	Save darkMode = true
Game level/score	Save level = 5, score = 90

← END Conclusion:

Writing to SharedPreferences means saving small user data like login status, settings, or preferences. It is done using `putString`, `putBoolean`, etc., and saved using `apply()` or `commit()`. This data stays even when the app is closed, making it perfect for remembering user choices.

5.4 Methods of editor class

◆ What is the Editor Class?

The Editor class is used to write data to SharedPreferences.

Whenever we want to save or change values in SharedPreferences, we use Editor.

```
SharedPreferences.Editor editor = prefs.edit();
```

✓ Common Methods of the Editor Class

◆ putString(String key, String value)

1. Saves a string value.
2. `editor.putString("username", "Rishi");`

◆ putInt(String key, int value)

1. Saves an integer value.
2. `editor.putInt("score", 100);`

◆ putBoolean(String key, boolean value)

1. Saves a true/false value.
2. `editor.putBoolean("isLoggedIn", true);`

◆ putFloat(String key, float value)

1. Saves a float value.
2. `editor.putFloat("rating", 4.5f);`

◆ putLong(String key, long value)

1. Saves a long number.
2. `editor.putLong("time", 1234567890L);`

◆ remove(String key)

1. Deletes a specific key and its value.
2. `editor.remove("username");`

◆ clear()

1. Deletes all data from SharedPreferences.
2. `editor.clear();`

◆ apply()

1. Saves data in the background (asynchronous).
2. Fast and recommended.
3. `editor.apply();`

◆ commit()

1. Saves data immediately and returns true if successful.
2. Slower than apply().
3. `boolean success = editor.commit();`

✓ Example:

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
SharedPreferences.Editor editor = prefs.edit();
```

```
editor.putString("username", "Rishi");
editor.putBoolean("isLoggedIn", true);
editor.putInt("score", 90);
editor.apply(); // Save the changes
```

✓ Summary Table:

Method	What It Does
putString()	Save string data
putInt()	Save integer
putBoolean()	Save true/false
putFloat()	Save decimal value
putLong()	Save long number
remove()	Delete one item
clear()	Delete all saved items
apply()	Save changes (fast, recommended)
commit()	Save changes (slower, gives result)



Conclusion:

The Editor class methods help you to add, edit, or delete data from SharedPreferences. Using methods like putString, remove, apply, etc., you can manage simple app data like settings or login info easily and effectively.

5.5 Reading from Shared Preference

◆ What is Reading from SharedPreferences?

It means:



Getting the stored values (like string, int, boolean) using their keys.

✓ Steps to Read from SharedPreferences:

◆ Step 1: Access SharedPreferences

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
```



"MyPrefs" is the name of the file.



MODE_PRIVATE means only your app can access this data.

◆ Step 2: Use get Methods to Read Values

```
String name = prefs.getString("username", "Guest");
```

```
boolean isLoggedIn = prefs.getBoolean("isLoggedIn", false);
```

```
int score = prefs.getInt("score", 0);
```

📌 The second value is the default value, used if the key is not found.

✓ Example Code:

```
SharedPreferences prefs = getSharedPreferences("MyPrefs", MODE_PRIVATE);
```

```
String name = prefs.getString("username", "Guest");
```

```
boolean isLoggedIn = prefs.getBoolean("isLoggedIn", false);
```

```
int score = prefs.getInt("score", 0);
```

```
Log.d("SharedPrefs", "Name: " + name);
```

```
Log.d("SharedPrefs", "Login Status: " + isLoggedIn);
```

```
Log.d("SharedPrefs", "Score: " + score);
```

✓ Available get Methods:

Method	Used to Get...
--------	----------------

getString()	Text (e.g. username)
-------------	----------------------

getInt()	Integer (e.g. score)
----------	----------------------

getBoolean()	True/false (e.g. login)
--------------	-------------------------

getFloat()	Decimal value
------------	---------------

getLong()	Long numbers
-----------	--------------

✓ Summary:

Task	Method Used
------	-------------

Get a string	getString("key", "default")
--------------	-----------------------------

Get a number	getInt("key", default)
--------------	------------------------

Get true/false value	getBoolean("key", default)
----------------------	----------------------------

🔙 Conclusion:

Reading from SharedPreferences helps you get stored values like username, login status, or app settings. It is useful when you want to remember user data, even after the app is closed.

Unit-6: Preserving and Saving data in Local Database

6.1 Introduction to SQLite

◆ What is SQLite?

- SQLite is a lightweight database used in Android to store structured data like:
 - User information
 - Login details

- Product list
- App history, etc.
- It is like a mini version of MySQL or other databases — but it works without a server.
- SQLite is an open-source SQL database that stores data to a text file on a device. Android comes with built-in SQLite database implementation.
- SQLite supports all the relational database features. In order to access this database, you don't need to establish any kind of connections for it like JDBC, ODBC, etc.

◆ Why Use SQLite in Android?

- 📱 Stores data inside your app
- 📶 Works offline
- 📁 Saves multiple records in tables
- 🆓 It's free and built-in with Android

◆ Key Features:

Feature	Description
Local Database	Data stored on device, not online
No Server Needed	Runs without MySQL or any server
Small & Fast	Lightweight and efficient
SQL Supported	You can use normal SQL queries

✓ Basic Structure of SQLite:

- Data is stored in tables (like Excel sheets)
- Each table has rows and columns

You can perform:

- INSERT (add data)
- SELECT (read data)
- UPDATE (change data)
- DELETE (remove data)

✓ Example Table: UserTable

ID	Name	Email
1	Rishi	rishi@gmail.com
2	Rana	rana@gmail.com

✓ How to Use SQLite in Android:

- Create a class that extends SQLiteOpenHelper
- Override onCreate() to create tables
- Use SQLiteDatabase methods to insert, read, update, and delete data

✓ Simple Code Example:

```
SQLiteDatabase db = openOrCreateDatabase("MyDB", MODE_PRIVATE, null);  
db.execSQL("CREATE TABLE IF NOT EXISTS users(id INTEGER, name TEXT)");  
db.execSQL("INSERT INTO users VALUES(1, 'Rishi')");
```

←
END Conclusion:

SQLite is a simple, powerful database used in Android apps to store data locally using SQL queries. It is best for apps that need to save and manage data without internet.

6.2 SqliteOpenHelper Class

◆ What is SQLiteOpenHelper?

- SQLiteOpenHelper is a helper class in Android used to:
 - Create a database
 - Update the database when needed
 - Manage tables inside the database
- It makes working with SQLite easier and more organized.
- The android.database.sqlite.SQLiteOpenHelper class is used for database creation and version management. For performing any database operation, you have to provide the implementation of onCreate() and onUpgrade() methods of SQLiteOpenHelper class.

✓ Why use SQLiteOpenHelper?

Instead of writing all the database code manually, you can:

- Create a custom class that extends SQLiteOpenHelper
- Easily manage tables and data
- Handle version updates of your database

✓ How to Use SQLiteOpenHelper?

You need to:

1. Create a class that extends SQLiteOpenHelper
2. Override two important methods:
 - onCreate(): Called once when the database is first created
 - onUpgrade(): Called when database version is changed

✓ Example Code:

```
public class MyDatabaseHelper extends SQLiteOpenHelper {  
  
    // Constructor  
    public MyDatabaseHelper(Context context) {
```



```

        super(context, "MyDB", null, 1); // DB name: MyDB, version: 1
    }

    // Called when the database is created
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT)");
    }

    // Called when database version is changed
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS users"); // Delete old table
        onCreate(db); // Create new one
    }
}

```

✓ Explanation of Code:

Part	What it Does
<code>super(context, "MyDB", null, 1)</code>	Initializes the database
<code>onCreate()</code>	Creates the table when app runs first time
<code>onUpgrade()</code>	Updates database if version is changed

✓ How to Use in Your App:

```

MyDatabaseHelper dbHelper = new MyDatabaseHelper(context);
SQLiteDatabase db = dbHelper.getWritableDatabase();

```

Now you can insert, read, update, or delete data using db.

✓ Advantages of SQLiteOpenHelper:

- Saves time and reduces code
- Automatically handles database creation
- Helps manage version upgrades
- Keeps database logic in one place

🔙 Conclusion:

SQLiteOpenHelper is a built-in class in Android that helps you create, manage, and update your SQLite database easily. You just override `onCreate()` and `onUpgrade()` methods, and Android handles the rest.

6.3 SQLite Methods - ExecSQL, Rawquery, Insert, Update, Delete

ExexSQL

◆ What is execSQL()?

- execSQL() is a method used in Android to run SQL queries (like CREATE, INSERT, DELETE, etc.) on an SQLite database.
- It is part of the SQLiteDatabase class.

✓ Purpose:

- To execute SQL commands that do not return results
- (Like: CREATE TABLE, INSERT, UPDATE, DELETE)

✓ Syntax:

```
db.execSQL("SQL Query");
```

★ db is your SQLiteDatabase object.

✓ Common Uses:

Task	Example
------	---------

Create Table	db.execSQL("CREATE TABLE users(id INTEGER, name TEXT)")
--------------	---

Insert Data	db.execSQL("INSERT INTO users VALUES(1, 'Rishi')")
-------------	--

Update Data	db.execSQL("UPDATE users SET name='Raj' WHERE id=1")
-------------	--

Delete Data	db.execSQL("DELETE FROM users WHERE id=1")
-------------	--

Drop Table	db.execSQL("DROP TABLE IF EXISTS users")
------------	--

✓ Example:

```
SQLiteDatabase db = openOrCreateDatabase("MyDB", MODE_PRIVATE, null);
```

```
// Create a table
```

```
db.execSQL("CREATE TABLE IF NOT EXISTS users(id INTEGER, name TEXT);
```

```
// Insert data
```

```
db.execSQL("INSERT INTO users VALUES(1, 'Rishi');
```

```
// Update data
```

```
db.execSQL("UPDATE users SET name='Raj' WHERE id=1");
```

```
// Delete data
```

```
db.execSQL("DELETE FROM users WHERE id=1");
```

✓ Important Notes:

- execSQL() does not return any data/result.

- It is used only for commands that change the database.
- If you want to read data, use:
- `rawQuery()` (for SELECT queries)

✓ Advantages:

- Easy to use
- Good for basic operations
- Works well with all types of SQL commands (except SELECT)

← Conclusion:

The `execSQL()` method in Android SQLite is used to run SQL commands that change the database like creating tables, inserting data, deleting records, etc. It's a simple way to work with databases in Android apps.

Rawquery

◆ What is `rawQuery()`?

- `rawQuery()` is a method in Android used to run SELECT queries in SQLite and get data (results) from the database.
- It returns a `Cursor`, which lets you read rows and columns from the result.

✓ Purpose:

- To read data from the database
- Works with SELECT statements only

✓ Syntax:

```
Cursor cursor = db.rawQuery("SELECT * FROM table_name", null);
```

✦ `db` is your `SQLiteDatabase` object

✦ `null` is used when there are no selection arguments

✓ Example:

```
SQLiteDatabase db = openOrCreateDatabase("MyDB", MODE_PRIVATE, null);
```

```
// Assume a table already exists with some data
```

```
Cursor cursor = db.rawQuery("SELECT * FROM users", null);
```

```
// Loop through the results
```

```
if (cursor.moveToFirst()) {
```

```
    do {
```

```
        int id = cursor.getInt(0);           // Get ID (1st column)
```

```
        String name = cursor.getString(1);   // Get Name (2nd column)
```

```

        Log.d("UserData", "ID: " + id + ", Name: " + name);
    } while (cursor.moveToNext());
}
cursor.close(); // Always close the cursor after use

```

✓ What is a Cursor?

- A Cursor is like a pointer that moves through each row of the result.

Cursor Method	Use
moveToFirst()	Move to first row
moveToNext()	Move to next row
getString(index)	Get string value at column
getInt(index)	Get integer value at column
getCount()	Total number of rows returned

✓ Use Case Example:

Table: users

ID	Name
1	Rishi
2	Rana

`rawQuery("SELECT * FROM users", null)` will return both rows, and you can read them using a Cursor.

✓ When to Use `rawQuery()`:

- When you want to read data
- When using custom SELECT queries
- When you need to loop through multiple results

← END Conclusion:

- `rawQuery()` is a method used in Android to fetch data from SQLite database using a SELECT query. It gives results in a Cursor, which helps you read each row one by one.
- It is very useful when you want to display stored data in your app.

Insert

◆ What is `insert()`?

- `insert()` is a method used in Android to add (insert) new data into a SQLite database table.
- Instead of writing a full SQL query, you use `insert()` with key-value pairs (called ContentValues).

✓ Purpose:

- To add a new row into a table with safe and easy code.

✓ Syntax:

```
long rowId = db.insert(tableName, nullColumnHack, contentValues);
```

Parameter	Meaning
tableName	The name of the table
nullColumnHack	Usually set to null
contentValues	A map (key-value) of column & value

✓ Steps to Use:

1. Create a ContentValues object
2. Put your data using put()
3. Call insert() on the database

✓ Example:

```
// Step 1: Get writable database
SQLiteDatabase db = dbHelper.getWritableDatabase();

// Step 2: Create content values
ContentValues values = new ContentValues();
values.put("name", "Rishi");
values.put("age", 22);

// Step 3: Insert data
long result = db.insert("users", null, values);

// Optional: Check result
if (result == -1) {
    Log.d("DB", "Insert Failed");
} else {
    Log.d("DB", "Insert Success, Row ID: " + result);
}
```

✓ What is ContentValues?

- It's a special class that works like a map of data.

Example:

```
values.put("column_name", "value");
```

✓ Output Table: users

ID	Name	Age
1	Rishi	22

✓ Advantages of insert():

- Safe from SQL injection
- No need to write raw SQL queries
- Easy to use and understand
- Returns row ID if successful

←
END Conclusion:

The insert() method in Android is used to add new data into a SQLite database table. You just use ContentValues to store data and call insert() to save it. It's clean, safe, and simple to use.

Update

◆ **What is update()?

- The update() method is used to modify or change existing data in a table of the SQLite database in Android.
- Instead of writing an SQL query, you can safely update values using key-value pairs (ContentValues).

✓ Purpose:

To change the data of one or more rows in a table based on a condition (like matching ID or name).

✓ Syntax:

```
int rowsAffected = db.update(  
    tableName,  
    contentValues,  
    whereClause,  
    whereArgs  
);
```

Parameter	Description
tableName	Name of the table
contentValues	New data (key-value pairs)
whereClause C	ondition (like id = ?)
whereArgs	Actual value for the ? (like ["1"])

✓ Example:

```
SQLiteDatabase db = dbHelper.getWritableDatabase();

ContentValues values = new ContentValues();
values.put("name", "Rishi");

// Update the row where id = 1
int result = db.update("users", values, "id = ?", new String[]{"1"});

if (result > 0) {
    Log.d("DB", "Update Successful");
} else {
    Log.d("DB", "Update Failed");
}
```

✓ What's Happening?

The name will be changed to "Rishi" where id = 1.
update() returns how many rows were updated.

✓ Before and After Table:

ID	Name
1	Rishi27
2	Rana

→ After running update() where id = 1:

ID	Name
1	Rishi
2	Rana

✓ Benefits of update():

- Safe from SQL injection
- Easy to use
- No need to write raw SQL queries
- You can update one or many rows at once

←
END Conclusion:

The update() method in SQLite is used to change existing records in a database table. It uses ContentValues to pass the new data and a condition (WHERE) to target the correct row(s). It's a secure and simple way to update data in Android apps.

DELETE

◆ What is delete()?

- The delete() method is used to remove data (rows) from a table in the SQLite database in Android.
- You can use it to delete specific records based on a condition (like ID = 1).

✓ Purpose:

To delete existing data from the database safely and easily.

✓ Syntax:

```
int rowsDeleted = db.delete(  
    tableName,  
    whereClause,  
    whereArgs  
);
```

Parameter	Meaning
tableName	Name of the table
whereClause	Condition (like id = ?)
whereArgs	Actual values for the ? placeholders

✓ Example:

```
SQLiteDatabase db = dbHelper.getWritableDatabase();  
  
// Delete user where id = 1  
int result = db.delete("users", "id = ?", new String[]{"1"});  
  
if (result > 0) {  
    Log.d("DB", "Delete Successful");  
} else {  
    Log.d("DB", "Delete Failed");  
}
```

✓ What's Happening?

- It will remove the row from the users table where id = 1.
- result tells how many rows were deleted.

✓ Before and After Table:

ID	Name
1	Rishi
2	Rana

→ After delete() where id = 1:

ID	Name
2	Rana

✓ Delete All Rows:

```
db.delete("users", null, null);
```

This deletes all rows in the users table!

✓ Advantages of delete():

- Easy and safe to use
- Avoids writing raw SQL
- Helps remove unwanted data
- Prevents SQL injection

← Conclusion:

The delete() method in SQLite is used to remove specific or all rows from a table in Android. It's clean, secure, and useful when you want to delete data from your app's database.

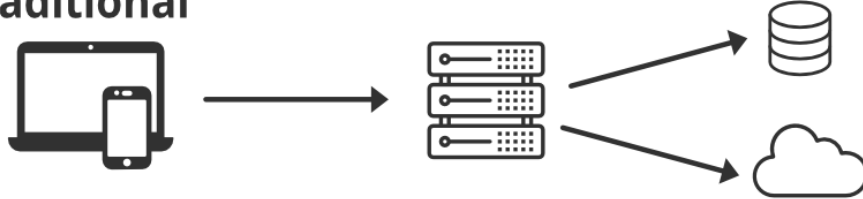
Unit-7: Handling Data with Firebase

7.1 Introduction to Firebase

◆ What is Firebase?

- Firebase is a platform by Google that helps developers build and manage mobile and web apps easily.
- It simplifies app development by offering a secure and efficient backend, eliminating the need for server-side programming. Firebase supports multiple platforms, including Android, iOS, web, and Unity, making it a versatile solution for developers.
- It provides tools like real-time database, authentication, cloud storage, hosting, analytics, and much more — all in one place.

Traditional



Firebase



✓ Purpose of Firebase:

- To help developers create powerful apps faster
- To reduce the need for complex backend coding
- To provide ready-made tools like:
 - Login system
 - Cloud database
 - Notifications
 - Crash reporting

✓ Firebase Works On:

- Android
- iOS
- Web apps

✓ Key Features of Firebase:

Feature	Use
◆ Firebase Realtime Database	Stores and syncs data in real-time
◆ Firebase Authentication	Login system using email, phone, Google, etc.
◆ Firebase Cloud Firestore	Advanced cloud database (NoSQL)
◆ Firebase Cloud Messaging	Send push notifications to users
◆ Firebase Storage	Upload and store images, videos, and files
◆ Firebase Hosting	Host your website easily
◆ Firebase Analytics	Track user activity in your app
◆ Firebase Crashlytics	Find and fix app crashes quickly

✓ Why Developers Love Firebase:

- Easy to use

- Saves time and effort
- Real-time updates
- Works without writing full server code
- Free to start with (generous free plan)

✓ Example Use Case:

If you're building a chat app, Firebase can help you:

- Store chat messages (Realtime Database)
- Allow users to log in (Authentication)
- Send notifications for new messages (Cloud Messaging)

All without writing complex backend code.

← Conclusion:

Firebase is a powerful platform from Google that helps developers quickly build apps with real-time data, login systems, cloud storage, notifications, and more. It is beginner-friendly and widely used in Android and web development.

7.2 Firebase - Environmental Setup

◆ What is Firebase Environment Setup?

- Environment setup means getting everything ready to use Firebase in your Android project.
- It includes connecting your app with Firebase, and adding required files and dependencies.

✓ Steps to Set Up Firebase in Android Studio:

◆ Step 1: Go to Firebase Console

- Open <https://console.firebase.google.com>
- Click on "Add project"
- Enter a project name and create your Firebase project

◆ Step 2: Register Your App

- Click "Add app" and select Android
- Enter your app's package name (like com.example.myapp)
- Click Register App

◆ Step 3: Download google-services.json File

- After registering, Firebase will give you a file called google-services.json
- Download it and paste it into the app/ folder of your Android project

◆ Step 4: Add Firebase SDK to Your Project

In project-level build.gradle:

```

        classpath 'com.google.gms:google-services:4.3.15'
        // Add in dependencies block
In app-level build.gradle:
        apply plugin: 'com.google.gms.google-services'

        dependencies {
            implementation 'com.google.firebase:firebase-auth:22.3.0'    // Example
            dependency
        }

```

✓ Add other Firebase libraries like firebase-database, firebase-storage as needed

◆ Step 5: Sync the Project

- Click "Sync Now" in Android Studio to apply the changes.

✓ Firebase is Now Connected!

You can now use features like:

🔑 Firebase Authentication

🔄 Firebase Realtime Database

☁ Firebase Storage

🔔 Firebase Notifications

🏠 Conclusion:

Firebase Environment Setup is the process of connecting your Android app to Firebase by creating a project, downloading the config file, and adding Firebase libraries. Once done, you can use powerful features like login, database, and cloud storage easily.

7.3 Writing Data to the Firebase

◆ What does it mean?

Writing data to Firebase means saving information (like user name, email, messages, etc.) into the Firebase Realtime Database or Cloud Firestore.

✓ Steps to Write Data:

◆ Step 1: Add Dependency

In build.gradle (app):

```
implementation 'com.google.firebase:firebase-database:20.3.0'
```

◆ Step 2: Get Database Reference

```
FirebaseDatabase database = FirebaseDatabase.getInstance();
```

```
DatabaseReference myRef = database.getReference("users");
```

◆ Step 3: Write Data

You can write simple text like this:

```
myRef.setValue("Hello Firebase!");
```

Or write an object:

```
DatabaseReference myRef = database.getReference("users");
```

```
myRef.child("user1").setValue("Rishi");
```

Write user data using a model class:

```
public class User {  
    public String name;  
    public int age;  
  
    public User() {} // Empty constructor required  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
// In activity  
User user = new User("Rishi", 22);  
myRef.child("user2").setValue(user);
```

✓ Output in Firebase Database:

```
"users": {  
  "user1": "Rishi",  
  "user2": {  
    "name": "Rishi",  
    "age": 22  
  }  
}
```

✓ Result:

Data is now stored in the cloud and can be accessed in real-time from any device using the app.

✓ Advantages:

- Real-time sync across devices
- Easy to use

- No need to write backend code
- Works even with slow internet (offline support)

🔙 Conclusion:

Writing data to Firebase means saving information into the cloud using the Realtime Database. You use `DatabaseReference` and `setValue()` to send data from your app. It is very easy and powerful for live apps like chats, forms, and user profiles.

7.4 Reading data

◆ What does it mean?

Reading data from Firebase means getting or fetching stored data from the Firebase Realtime Database so it can be shown in your app — like displaying user name, messages, etc.

✓ Steps to Read Data from Firebase:

◆ Step 1: Get Database Reference

```
FirebaseDatabase database = FirebaseDatabase.getInstance();
DatabaseReference myRef = database.getReference("users");
```

◆ Step 2: Use a Listener to Read Data

```
myRef.addValueEventListener(new ValueEventListener() {
    @Override
    public void onDataChange(DataSnapshot dataSnapshot) {
        // Read simple value
        String value = dataSnapshot.getValue(String.class);
        Log.d("Firebase", "Value is: " + value);
    }

    @Override
    public void onCancelled(DatabaseError error) {
        // Failed to read value
        Log.w("Firebase", "Failed to read value.", error.toException());
    }
});
```

✓ This reads the value from the "users" node and shows it in the Logcat.

◆ Example with Object (Model Class):

If your Firebase has this data:

```
"user1": {
  "name": "Rishi",
  "age": 22
```

```
}
```

Create a model class:

```
public class User {  
    public String name;  
    public int age;  
  
    public User() {}    // Required empty constructor  
}
```

Then read it:

```
myRef.child("user1").addListenerForSingleValueEvent(new ValueEventListener() {  
    @Override  
    public void onDataChange(DataSnapshot dataSnapshot) {  
        User user = dataSnapshot.getValue(User.class);  
        Log.d("Firebase", "Name: " + user.name + ", Age: " + user.age);  
    }  
  
    @Override  
    public void onCancelled(DatabaseError error) {  
        Log.w("Firebase", "Failed to read user.", error.toException());  
    }  
});
```

✓ Output:

Name: Rishi

Age: 22

✓ Advantages of Firebase Reading:

- Real-time updates (auto refresh)
- Simple code
- Can read text or objects
- Works with offline caching too



Conclusion:

Reading data from Firebase means using a DatabaseReference and a listener like addValueEventListener() to get the data stored in Firebase Realtime Database. It's a fast and simple way to display real-time data in Android apps.

Unit 8: Working with other Resources

8.1 Geocoding and reverse Geocoding

🔍 What is Geocoding?

🕒 Geocoding is the process of converting a place name or address into latitude and longitude.

👉 Example:

If you type "Statue of Liberty", geocoding gives:

📍 Latitude: 40.6892

📍 Longitude: -74.0445

💠 What is Reverse Geocoding?

🔄 Reverse Geocoding is the opposite — it converts latitude and longitude into a place name or address.

👉 Example:

If you have Lat: 40.6892, Long: -74.0445, reverse geocoding gives:

📍 "Statue of Liberty, New York, USA"

✓ Why is it useful?

Feature

Use

📍 Geocoding

Find coordinates to place markers on maps

🔄 Reverse Geocoding

Show user-friendly addresses in location-based apps

✓ Android Example Using Geocoder Class

To Geocode (get lat/long from location name):

```
Geocoder geocoder = new Geocoder(context, Locale.getDefault());
```

```
List<Address> addresses = geocoder.getFromLocationName("Surat, India", 1);
```

```
double lat = addresses.get(0).getLatitude();
```

```
double lon = addresses.get(0).getLongitude();
```

To Reverse Geocode (get address from lat/long):

```
Geocoder geocoder = new Geocoder(context, Locale.getDefault());
```

```
List<Address> addresses = geocoder.getFromLocation(21.1702, 72.8311, 1);
```

```
String address = addresses.get(0).getAddressLine(0);
```

✓ Real-life Uses:

- Cab apps like Uber (find pickup/drop location)
- Weather apps (show location-based weather)
- Delivery apps (get your address from GPS)
- Maps (place markers on searched locations)

✓ Requirements:

- Internet connection (for Geocoder)
- Location permission in Android app

←
END Conclusion:

- Geocoding = Address → Coordinates
- Reverse Geocoding = Coordinates → Address
- It is used in location-based Android apps to find and show user-friendly places or exact positions on a map.

8.2 Audio, Video and Using the Camera

◆ 1. Playing Audio in Android

Android provides a class called MediaPlayer to play audio files like .mp3.

✓ Example:

```
MediaPlayer mediaPlayer = MediaPlayer.create(context, R.raw.mysong);  
mediaPlayer.start();    // Starts playing the audio
```

◆ Common Functions:

Method	Use
start()	Start playback
pause()	Pause playback
stop()	Stop playback
release()	Free memory

✓ Audio files are usually stored in the res/raw folder.

◆ 2. Playing Video in Android

You can use VideoView to play videos from the app or internet.

✓ Example (from local file):

XML:

```
<VideoView  
    android:id="@+id/videoView"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"/>
```

Java:

```
VideoView videoView = findViewById(R.id.videoView);
```

```
videoView.setVideoURI(Uri.parse("android.resource://" + getPackageName() + "/"
+ R.raw.myvideo));
videoView.start();
```

✔ Video files are also stored in the res/raw folder.

◆ You can also use MediaController to show play/pause buttons:

```
MediaController mediaController = new MediaController(this);
videoView.setMediaController(mediaController);
```

✔ Summary Table:

Feature	Tool/Class Used	Purpose
🔊 Audio	MediaPlayer	Play audio files like songs
🎥 Video	VideoView, MediaPlayer	Play videos from local or online
📷 Camera	Intent + MediaStore	Capture photos or videos

✔ Real-Life Uses:

- Music apps (Spotify, Gaana)
- Video players (MX Player)
- Chat apps (photo/video capture)

←
END Conclusion:

Android makes it easy to play audio, show videos, and use the camera using built-in classes like MediaPlayer, VideoView, and Intent. These features are essential for modern apps involving media and user interaction.

8.2.1. Recording Audio and Video

◆ 3. Recording Audio in Android

Android uses MediaRecorder to record audio from the microphone.

✔ Steps:

1. Add permission in AndroidManifest.xml:

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

2. Java Code:

```
MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
recorder.setOutputFile("/sdcard/recorded.3gp");
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
```

```
recorder.prepare();
recorder.start();    // Start recording
```

To stop:

```
recorder.stop();
recorder.release();
```

🔊 Audio is saved as .3gp file.

◆ 4. Recording Video in Android

You can record video using the Camera Intent or MediaRecorder.

✓ Easy Way (Camera Intent):

```
Intent intent = new Intent(MediaStore.ACTION_VIDEO_CAPTURE);
startActivityForResult(intent, 101);
```

In onActivityResult():

@Override

```
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == 101 && resultCode == RESULT_OK) {
        Uri videoUri = data.getData();
        videoView.setVideoURI(videoUri);
        videoView.start();    // Play recorded video
    }
}
```

🎥 This will open the camera and let the user record a video.

✓ Summary Table:

Feature	Tool Used	Purpose
🎧 Play Audio	MediaPlayer	Play music or sounds
🎬 Play Video	VideoView	Play local or online videos
🔊 Record Audio	MediaRecorder	Record sound from microphone
🎥 Record Video	Intent / MediaRecorder	Record videos using camera

🔙 Conclusion:

Android provides simple classes like MediaPlayer, VideoView, and MediaRecorder to play and record audio/video. These features are commonly used in multimedia and social apps. Proper permissions and file handling are important to ensure smooth working.

8.2.2. Working with the Camera

◆ Using the Camera in Android

You can use the camera to take photos or record videos directly from your app.

✓ Example: Open Camera to Capture Image

```
Intent cameraIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
startActivityForResult(cameraIntent, 100);
```

✓ Get Result in onActivityResult():

@Override

```
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == 100 && resultCode == RESULT_OK) {
        Bitmap photo = (Bitmap) data.getExtras().get("data");
        imageView.setImageBitmap(photo);
    }
}
```

✓ Make sure to add camera permission in AndroidManifest.xml:

```
<uses-permission android:name="android.permission.CAMERA" />
```

And for Android 6.0+, also request it at runtime.

✓ Summary Table:

Feature	Tool/Class Used	Purpose
📷 Camera	Intent + MediaStore	Capture photos or videos

8.3 Push Notification

◆ What is Push Notification?

A Push Notification is a short message that pops up on the user's device screen — even when the app is not open.

📄 Example:

- "You have a new message" (from WhatsApp)
- "Today's weather is 32°C" (from Weather app)
- "Flash sale starts now!" (from a shopping app)

◆ How Does It Work?

1. Your app is connected to a Push Notification Server (like Firebase Cloud Messaging – FCM).
2. When an update or message is available, the server sends a push message to the user's device.
3. The notification appears in the status bar or notification tray.

◆ Technologies Used:

Tool	Purpose
◆ Firebase Cloud Messaging (FCM)	Send push notifications
◆ NotificationManager	Show notification in Android
◆ NotificationCompat.Builder	Create a custom notification

◆ Example Code:

```
NotificationCompat.Builder builder = new NotificationCompat.Builder(this,
"my_channel")
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle("New Message")
    .setContentText("Hello! You have a new message.")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT);
```

```
NotificationManagerCompat notificationManager =
NotificationManagerCompat.from(this);
notificationManager.notify(1, builder.build());
```

◆ Where Is It Used?

✓ Common apps that use push notifications:

- 📧 Chat apps (WhatsApp, Telegram)
- 🛒 E-commerce apps (Amazon, Flipkart)
- 📅 Calendar/reminder apps
- 🔊 News and social media apps

← END Conclusion:

Push Notifications are used to send alerts and updates to users even when the app is not open. It helps improve user engagement and keep users informed.